Final Year Project

PATS: A Priority-Aware Task Scheduling Algorithm for Multi-Access Edge Computing

Cian Boyle

A thesis submitted in part fulfilment of the degree of

BSc. (Hons.) in Computer Science

Supervisor: Professor Liam Murphy



UCD School of Computer Science
University College Dublin

April 26, 2026

Table of Contents

1 Project Specification 4

1.1 Project Specification 4

2 Introduction 5

3 Related Work and Ideas 7

3.1 Cloud Computing 7

3.2 Edge Computing 7

3.3 Multi-access Edge Computing (MEC) 8

3.4 Heterogeneous Task Challenges 8

3.5 Migration 9

3.6 Operating System Scheduling 10

3.7 Priority-Aware Algorithms 10

3.8 Priority Scoring Functions 11

3.9 iFogSim2 11

4 Design & Implementation 12

4.1 A Purpose-Built Scheduling Layer for iFogSim2 12

4.2 Priority Tasks 12

4.3 New Edge Server Entity 12

4.4 Abstract Scheduler 14

4.5 Edge-state Snapshot 14

4.6 Scheduler Metrics 15

4.7 Scheduling Algorithms 16

4.8 PATS Algorithm 16

4.9 Time Complexity Analysis of PATS 21

4.10 System Model 22

5 Simulation & Results Analysis 24

5.1 Verification Testing 24

5.2 Performance Testing 29
5.3 Temporal Arrival Patterns 43
6 Conclusions and Future Work 48
6.1 Conclusion 48
6.2 Future Work 48
A GitLab Repository & Simulation Videos 53

Abstract

The Multi-access Edge Computing (MEC) paradigm, an extension of the cloud computing paradigm, has become significantly more popular over the past number of years. By bringing computational resources closer to end users, it allows for latency-sensitive tasks to be performed with significantly less delay. However, due to the heterogeneous nature of modern-day computing tasks, challenges have arisen around task scheduling as tasks themselves can differ in priority, resource demands, and latency requirements. In a system with no perception of priority, time-sensitive tasks such as cardiovascular monitoring may be placed further back in a queue than some lower priority tasks such as a file backup. This project proposed a Priority-Aware Task Scheduling (PATS) algorithm, designed to improve task Quality of Experience (QoE) for high-priority, time-critical tasks while also including considerations so as to not sacrifice the QoE of tasks of lesser importance. The algorithm uses a priority scoring function to determine the order in which tasks are considered for scheduling. The algorithm also makes use of a priority-aware migration mechanism which preempts low-priority work when resources are saturated, freeing capacity for the timely execution of high-priority tasks. The iFogSim2 simulation framework was extended to support an additional dedicated scheduling layer to allow for comprehensive testing of the algorithm across various dimensions. Performance was evaluated against the baseline algorithm of First-Come First-Served (FCFS) across a range of configurations, including different task volumes, priority splits, cloud latencies, and temporal arrival patterns. Results showed that PATS consistently outperformed FCFS, through its mechanisms of protecting high-priority QoE even under heavy system load, while still keeping low-priority tasks at acceptable service levels. Energy consumption was also considered across varying numbers of provisioned edge servers to monitor the rate at which QoE can be improved with increasing energy usage. Findings showed that PATS produced superior task QoE using fewer edge servers than FCFS, presenting a useful trade-off between QoE and energy consumption for real-world deployment scenarios.

Chapter 1: Project Specification

1.1 Project Specification

One core objective of this project is to design and implement a purpose-built scheduling framework for Multi-access Edge Computing (MEC) environments within the iFogSim2 simulator. The second core objective is to design a new Priority-Aware Task Scheduling algorithm and implement it using the new framework for iFogSim2. The final objective is to evaluate the new priority-aware algorithm across multiple dimensions against FCFS, which serves as the baseline approach.

1.1.1 Core Requirements

1. Gain a deep, working understanding of the **iFogSim2** simulator and its event-driven architecture.
2. Design and implement a purpose-built, extensible scheduling layer for iFogSim2, to allow future studies to implement and compare other scheduling algorithms.
3. Implement a baseline FCFS scheduling algorithm using the new framework for scheduler analysis.
4. Design the Priority-Aware Task Scheduling (PATS) algorithm and implement it within iFogSim2.
5. Implement a metrics collection system as part of the new scheduling framework
6. Conduct verification testing to validate scheduling decisions (priority ordering, placement strategies, preemption, cloud offloading) behave as intended.

1.1.2 Advanced Requirements

1. Conduct performance testing comparing PATS and FCFS across multiple dimensions with different numbers of configured edge servers.
2. Evaluate algorithm robustness under realistic temporal arrival patterns analysing both QoE outcomes and operational metrics.
3. Analyse the energy-efficiency trade-off between the number of edge servers deployed and QoE achieved.

Chapter 2: **Introduction**

In recent years, cloud computing has seen rapid growth in processing and storing large amounts of data which provides a large number of services to users all around the world. However, one identified issue regards the distance between cloud data centres and end-users, which can sometimes stretch as far as one continent to another. This led to the development of Multi-Access Edge Computing (MEC), which essentially brings cloud computing resources closer to where the data is produced, which as a result can significantly minimise round-trip time and load on the cloud.

MEC sits between the user and the cloud, enabling task offloading for workloads that are either too computationally expensive to run on the user's device or too time-critical to tolerate the latency of cloud offloading. The rapid growth of IoT [1] has introduced a wide range of heterogeneous tasks that are candidates for edge execution. Tasks can be classified by importance which can be based on latency requirement, computational need or even just the type of application which generates it. Typically they can range on a spectrum from high-priority (urgent, critical application) to low-priority (non-urgent, non-critical application). Representative examples include collision detection in autonomous vehicles as high-priority [2], with AR/VR streaming as medium-priority and sensor data uploads as low-priority [3].

Due to the heterogeneous nature of tasks which can execute on the edge, it is essential to ensure that tasks which have an intrinsically higher priority than others are processed first. A system that has no perception of priority and is overloaded with low-priority, non-urgent tasks could result in time-critical, high-priority tasks waiting to be executed, which for certain applications, may be unacceptable. Mitigating this issue was a key goal of this project through the proposal of the Priority-Aware Task Scheduling (PATS) algorithm. While previous work attempted to combat the issue of heterogeneous task scheduling within MEC environments, they are often one-dimensional or rely on computationally intensive deep learning techniques. The goal behind the design of the newly proposed algorithm was to ensure that the highest-priority of work always received the best levels of service through a heuristic approach. This was implemented through various techniques such as dynamic task scoring, cloud offloading and priority-based preemption, the latter of which posed the greatest difficulty as lower-priority work still needed to receive acceptable levels of service.

The iFogSim2 simulator was chosen for this project due to its event-driven architecture, which enabled realistic modelling of asynchronous task dispatch and completion across a distributed set of edge servers. Although an initial challenge arose due to specific modelling of pre-existing classes within the simulator, this presented the opportunity to extend the framework by incorporating a dedicated scheduling layer for scheduling analysis. Using the scheduling layer within the framework, the newly proposed PATS algorithm was implemented, along with a baseline scheduling approach - First-Come First-Served (FCFS).

The implemented algorithms were then heavily tested across a variety of different regimes, ranging from verification testing to validate algorithmic correctness, performance testing to consider a multitude of configurations and finally temporal arrival patterns to examine performance in real-world deployment scenarios. Energy efficiency was another key consideration of this study during testing. Studying the trade-off of how additional edge servers impact overall system performance in terms of improving task quality of experience (QoE) is crucial for network operators and was an integral part of this work. PATS was observed to consistently outperform FCFS across all tested configurations, achieving maximum high-priority QoE, notably with fewer edge servers than FCFS required.

The remainder of this thesis is structured as follows. Chapter 3 reviews related work in cloud and edge computing, task heterogeneity, migration, and existing priority-aware scheduling approaches. Chapter 4 presents the design and implementation of the new scheduling layer within iFogSim2, along with the PATS algorithm itself. Chapter 5 details the simulation setup and evaluates PATS against the FCFS baseline across verification, performance, and temporal arrival pattern tests. Chapter 6 concludes the thesis and outlines directions for future work.

Chapter 3: Related Work and Ideas

This chapter includes the discussion of research, which was carried out on previous work and ideas relevant to this project. Firstly, cloud computing is outlined followed by the discussion of the subsequent need for edge computing. Discussion then moves to analysing the heterogeneous nature of work within a MEC environment, which then leads to the consideration of priority-based migration, a crucial component of this project. Following the consideration of heterogeneity, analyses shift to previous work which attempts to combat the issues which arise. Finally, a brief introduction to the event-driven simulator, iFogSim2 is included, which is the tool extended and utilised for this study.

3.1 Cloud Computing

The NIST definition [4] of cloud computing is read as follows:

"a model for enabling ubiquitous, convenient, on demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction."

A key characteristic of this paradigm being elasticity [4], has enabled computing resources to be dynamically provisioned and released to meet varying workload demands which has allowed organisations to improve the scalability of their services and deal with bursts in resource demands [5]. As a result, organisations have gone from a Capital Expenditure (CapEx) model to an Operational Expenditure (OpEx) model [6], which avoids massive upfront costs for computing infrastructure. While this widespread shift has led to so many organisations utilising cloud services, due to the significant distances from end-users to the cloud itself, time-sensitive work going to the cloud may be inadequate and as a result has led to the rise of the edge computing paradigm.

3.2 Edge Computing

Edge computing extends the cloud paradigm by bringing computing infrastructure closer to the end user [7]. Rather than transmitting all data to distant data centres, processing occurs at the network edge, right where data is generated. This as a result reduces response times, improves processing efficiency, and lowers bandwidth demand on the core network [8]. These properties make edge computing well suited to latency-sensitive workloads that cannot tolerate the round-trip delay of cloud execution. However, edge nodes operate as part of a distributed architecture with constrained resources, which introduces new challenges when handling the heterogeneous mix of tasks that arrive at the network. This is a core issue of this study, which will be explored in detail.

3.3 Multi-access Edge Computing (MEC)

Multi-access Edge Computing (MEC) applies the edge computing paradigm within mobile and wireless access networks, placing compute, storage, and networking resources at base stations or access points [9, 10]. Figure 3.1 illustrates a typical MEC deployment.

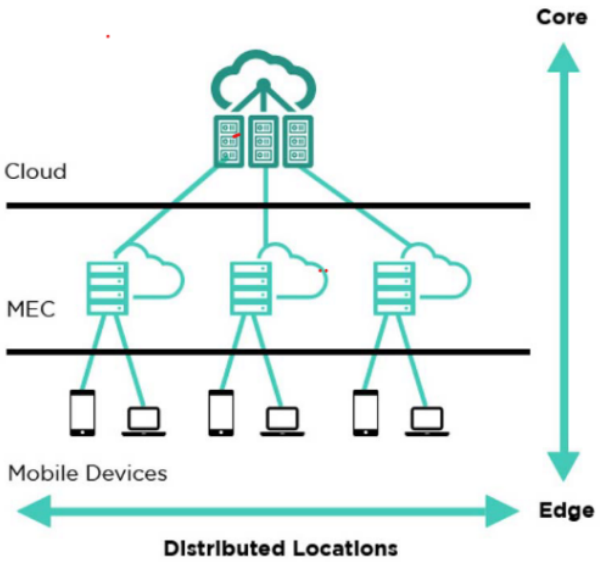


Figure 3.1: Architecture of Multi-access Edge Computing from [11]

By situating compute resources within one hop of the device, MEC allows mobile devices to offload resource-intensive tasks without incurring the latency penalty of reaching a distant cloud. This frees on-device compute for lighter workloads while still meeting the ultra-low latency requirements of time-critical applications such as autonomous-vehicle collision detection, continuous heart-rate monitoring, and AR/VR rendering. This is a great benefit of the paradigm, however, this benefit is bounded by the limited resources available at the edge. As IoT continues to grow, leading to ever increasing traffic volumes, MEC nodes face sustained contention for compute capacity. This issue paired with the fact that not all traffic shares the same priority or latency requirements, presents a real issue. Effectively allocating scarce edge resources across a heterogeneous workload is the problem this study aims to address.

3.4 Heterogeneous Task Challenges

Due to advancements in recent years, there are a vast array of IoT devices [12], all of which could be connected to the same edge network. For example, we could have wearable health devices, smart sensors, surveillance cameras which are all generating data and requesting computing resources for processing. Each of these instances generates tasks which are inherently different, having different data sizes, computational needs, and latency requirements. [13]

This is a challenge for naive scheduling algorithms such as an approach like "First-Come First-Served" scheduling [14]. As it allocates resources to tasks in order of arrival, it is unable to differentiate between intrinsically time-critical work such as health-monitoring tasks and flexible, background tasks such as file backups. Abstracting away the specifics of each task and its originat-

ing application, health-monitoring can be treated as high-priority and file-backups as low-priority. In a scenario in which a long duration, low-priority task arrives just before a critical latency-constrained task, a FCFS scheduler will allocate resources for the low-priority task first, before considering the higher priority task. This results in latency-sensitive applications experiencing delays due to background work taking up computation time, which reduces an end user's Quality of Experience (QoE) and in the context of safety-critical applications, risks serious harm.

Another challenge is regarding task placement, arising due to the dynamic nature of task arrivals into a network. Tasks begin and finish execution at different times, meaning edge resources become available unpredictably. An approach such as Round-Robin is a static placement strategy [15] and does not alter or adapt its behavior based on current system state, and instead, simply cycles through some fixed sequence. For example, if a new task arrives while an existing node becomes free, Round-Robin continues following sequence and may result in the task being assigned to a busy node instead of taking advantage of the newly available one. Due to the algorithm ignoring real-time changes in task load and node capacity, it can lead to suboptimal performance. Within dynamic, heterogeneous IoT environments it is crucial for schedulers to consider dynamically changing workloads, heterogeneous node capabilities, and varying system conditions to achieve efficient performance [16, 17]. This points to the need for adaptive, priority-aware scheduling and placement strategies to improve upon simpler approaches such as FCFS and Round-Robin.

3.5 Migration

Migration is the act of moving an executing task from one edge server to another, which is already a well-researched area in the field [18]. It is a primary method to ensure that there is no loss of performance due to overloaded or undesirable nodes in a network, but also to improve resource utilisation overall. The cloud can also be considered as an option to offer superior computational resources provided the round-trip time does not compromise latency, as also discussed in [18].

Although task migration is a well-studied area, much of the current literature focuses on mobility [19] and resource utilisation rather than on using migration to improve service for high-priority tasks. When a critical task enters a network demanding immediate computing resources, an adjustment to the current allocation may be required. If regular background traffic already occupies all available edge resources, a newly arrived latency-critical task must share compute with lower-priority work, degrading its performance. When resources are fully saturated and no placement option is possible, the task is forced to wait in the queue or even be offloaded to the cloud, extending its response time. However, migrating lower-priority work away from the edge to free resources for more important tasks allows the critical tasks to receive the highest quality of service, which can be crucial for certain applications.

Priority-driven task migration remains largely unexplored in current literature, leaving a clear research gap, especially when considering environments with tasks of varying priority. These types of environments require dynamic preemption and resource allocation to ensure that high-priority tasks receive low latency service but also ensure low disruption on other ongoing tasks in the network.

3.6 Operating System Scheduling

When considering priority but also fairness within the domain of scheduling, the Linux Completely Fair Scheduler (CFS) [20], is a well-established approach to balancing the two for CPU scheduling. It essentially defines how much of a time-share, individual processes get of the CPU. To address the issue of priority, processes are assigned 'Nice Values'. This value is mapped to a weight through an exponential table, where lower Nice Values correspond to higher weights, resulting in processes receiving more CPU time. Processes' total running time is tracked and represented by its 'Virtual Runtime'. This Virtual Runtime is updated each time a process runs, using the following function:

$$\text{vruntime} = \text{vruntime} + \Delta t_{\text{exec}} \times \frac{w_0}{w_i} \tag{3.1}$$

where Δt_{exec} is the actual wall-clock time the process executed since the last update, w_0 is the weight of a process at Nice Value 0 (the default baseline), and w_i is the weight of the current process. This means higher-priority tasks accumulate Virtual Runtime at a slower rate per unit of actual CPU time consumed, as their larger weight reduces the scaling factor. The scheduler then always selects the process with the minimum Virtual Runtime from a red-black tree, resulting in higher-priority tasks receiving a greater share of CPU allocation as their slower-growing Virtual Runtime keeps them at the front of scheduling decisions.

However, CFS is mainly concerned with the scheduling of processes on a single operating-system kernel, across various CPU cores and is therefore not directly applicable to the domain of scheduling within MEC systems. The challenges of working with heterogeneous edge servers, potential cloud-offloading and priority-based migration are something that simply does not apply to the area of CPU scheduling. That said, the core idea of utilising scoring systems to determine scheduling is a valuable concept. Being able to prevent starvation through accumulated total running time is itself one-dimensional, but, along with a weight-based multiplier derived from priority to scale the rate of accumulation, allows the scheduler to make much more informed decisions. The idea of scaling process or 'task' importance scores by priority weights is something which is further explored in this work but specifically applied to the field of MEC.

3.7 Priority-Aware Algorithms

The notion of encoding priority into scheduling algorithms is an area which has been researched before. A notable example is the multi-queue strategy proposed by Fahad et al. in [21], which classifies tasks as short-lived or long-lived and schedules them accordingly. This simple yet thoughtful separation attempts to decrease task starvation, especially for long-duration tasks which are often at risk of being preempted by shorter, more latency-sensitive tasks.

Although this approach is appealing in the sense that it combats the issue of long-duration task starvation, it is quite narrow in execution. It essentially only considers task duration as the only factor when considering fairness and in a sense assumes that duration implies some sort of importance. A situation may arise such that we have a long-duration task of relatively minor importance, but also a short-duration task which is of high importance, ultimately causing the distinction between low and high-priority tasks to break down. The multi-queue approach certainly addresses the issue of starvation but does not address the issue of task heterogeneity. This highlights an issue in current research as "priority-aware" algorithms only consider one dimension of the problem, in this case being duration.

3.8 Priority Scoring Functions

Other research attempts to handle the issue of heterogeneous tasks in a network environment using priority scoring mechanisms. The work in the Computing Power Network (CPN) paper by Xie et al. [22], presents a scoring function which evaluates tasks being submitted to the network on various features of the tasks such as deadlines, waiting times and an external factor of the task - load on the router in which it is currently located. This work demonstrates the value of utilising multiple factors in evaluating task priority and demonstrates how using these evaluations can drive better scheduling decisions rather than only using single dimensions.

However, the paper primarily focuses on integrating the scoring function inside of a deep learning framework which handles task placement inside an abstract CPN model which the paper refers to. This limits the applicability of this approach to actual MEC networks. The paper also makes no discussion regarding model training computational cost overhead or even the abstract system's performance inside real-world scenarios, leading to further questions around its usefulness in practical MEC scenarios.

Nonetheless, the paper provided a sound conceptual idea in the use of the priority scoring functionality. Although integrated inside a complex deep learning framework, it still leaves the opportunity to explore its concepts inside a more heuristic powered domain. This leaves an area of research to be explored to evaluate how a priority-scoring powered algorithm can be integrated into a real-world MEC environment which is constrained by ever changing tasks priorities and requirements.

3.9 iFogSim2

As mentioned previously, there are a plethora of potential challenges to study in the domain of fog and edge computing. To study and analyse issues in greater detail, a selection of simulators have emerged which allow researchers to implement and study extensions for fog and edge computing environments. One such simulator is iFogSim2, which is the simulator employed for this project [23].

iFogSim2 itself is the successor to iFogSim [24], which is a Java-based simulation toolkit to help analyse situations in a controlled environment. Although it has many advanced integrations into the framework already, including microservice placement policies, mobility management, and dynamic clustering mechanisms, a priority-aware task scheduling algorithm is yet to be integrated. Furthermore, a general framework to analyse scheduling and placement algorithms is not yet included in the simulator. The simulator does not include traditional task scheduling algorithms such as First-Come-First-Served, or Shortest-Job-First, which presents both a challenge and an opportunity for this project. This project investigated extending the simulator to include a configurable scheduling layer to allow researchers to integrate new algorithms, create network simulations, run simulation tests with different scheduling policies, and collect metrics to analyse for further research. As part of this work, a baseline scheduling algorithm was also implemented to enable comparative analysis of this project's proposed algorithm.

Chapter 4: Design & Implementation

This chapter outlines the approach to designing and implementing both a new scheduling framework and the Priority-Aware Task Scheduling (PATS) algorithm into the iFogSim2 simulation toolkit. This project makes two contributions to the field of Multi-Access Edge Computing (MEC). Firstly, a purpose-built, and extensible scheduling layer into iFogSim2 which makes use of its event driven architecture to allow other researchers to implement their own scheduling algorithms and test them within a MEC environment with minimal effort. The second contribution being the PATS algorithm itself, which makes use of the newly developed scheduling framework to implement a priority-aware scheduler which has various innovative features such as priority scoring, priority driven migration and dynamic cloud offloading capabilities. This chapter goes into detail on the new system architecture and the new components utilised by the scheduling layer, the PATS scheduling logic, and the system model used for testing.

4.1 A Purpose-Built Scheduling Layer for iFogSim2

The new scheduling layer was designed with the intention that scheduling policy is all which researchers need to consider during implementation. The infrastructure for submitting tasks, managing edge server state, assigning tasks to edge servers or to the cloud, and capturing key system metrics are all provided within the framework itself. This simplifies the comparison of algorithms as when conducting experiments, the only thing which must change is the core scheduling strategies.

4.2 Priority Tasks

iFogSim currently allows users to set up devices such as sensors and actuators which generate tuples, however as a focus of this study is to introduce a new scheduling layer to test and compare task scheduling logic within MEC environments, a "PriorityTask" class, appropriate for the framework was created. Tasks are instantiated with task requirements which include MIPS, RAM, and a latency requirement. As this work focuses on priority-aware scheduling, a "TaskPriority" enum is also provided in the class to differentiate tasks of low-priority to tasks of high-priority. The class abstracts away low-level details and provides an appropriate interface to represent work to be completed in a MEC environment.

4.3 New Edge Server Entity

iFogSim's current implementation includes an inbuilt "FogDevice" class which can be used to represent a physical or virtual computing node within a MEC environment. It models complex behaviours relating to microservice placement and mobility which were not necessary for this

study and previous implementations have made it difficult to have fine-grained control over task execution.

To resolve this issue, the new "EdgeServer" entity which processes the new "PriorityTask" was created and provides a more accessible and relevant abstraction for a compute node which will be utilised for this project. The "EdgeServer" class extends CloudSim's SimEntity [25] and can be configured with processing capacity (MIPS), available RAM and network latency which are dynamically updated as tasks arrive and complete, similar to the native "FogDevice" within iFogSim2 [23]. Task processing follows a processor-sharing model [26] in which available MIPS are divided equally among active tasks, with a context-switching overhead [27] which degrades server efficiency as the number of concurrently running tasks increases, to provide a more realistic model of how server performance decreases with increased load. The number of MIPS allocated per task in one work cycle is given by the equation below:

$$MIPS_{task} = \frac{MIPS_{server}}{n} \times E(n) \tag{4.1}$$

where $MIPS_{server}$ represents the total processing capacity of the edge server, n is the number of concurrently executing tasks, and $E(n)$ is the efficiency factor which captures the reduction in effective processing capacity caused by context-switching overhead.

Server efficiency is calculated as a function of the number of concurrent tasks and is defined as:

$$E(n) = \frac{1}{1 + \alpha(n-1)} \tag{4.2}$$

where n is the number of active tasks and α represents the context-switch overhead coefficient which determines the rate at which server efficiency decreases as task concurrency increases.

Below is a graph illustrating how server efficiency decreases as the number of concurrently executing tasks increases, with $\alpha = 0.1$.

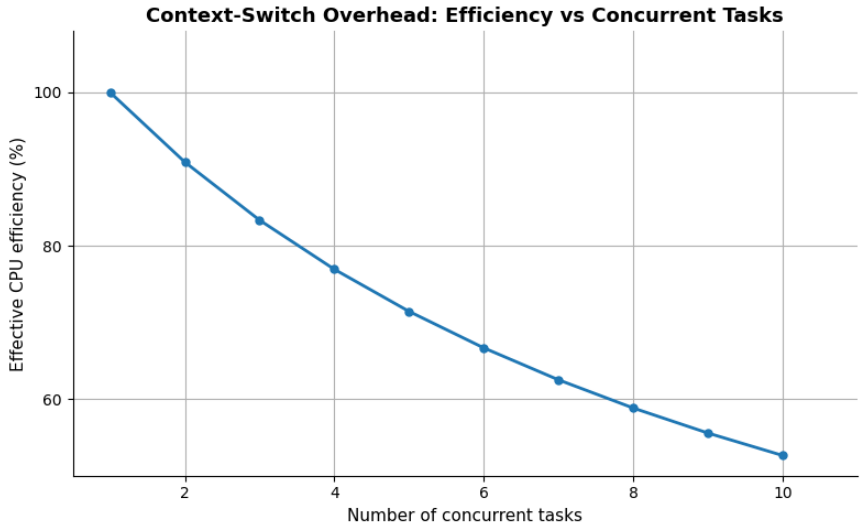


Figure 4.1: Server Efficiency Decrease with Increased Load

A linear power model tracks energy consumption based on CPU utilisation [28], consistent with the "FogDevice" implementation in iFogSim2 [23], where power consumption is modeled as a function of CPU utilisation derived from system load. The server continuously monitors its own load state, where IDLE indicates no tasks are currently executing, LIGHT means the server has

some tasks but is operating above minimum efficiency, and HEAVY means it is operating below minimum efficiency. The server exposes metrics like queue depth, estimated completion time, and round-trip time so the scheduler can make informed scheduling decisions.

Communication with the scheduler uses CloudSim's event-passing system; the server reports task completions and rejections back to scheduler, forming a feedback loop.

4.4 Abstract Scheduler

The AbstractScheduler class was a core new addition to the simulator and represents the central piece which integrates the other new components into the simulation framework. It extends the CloudSim, "SimEntity" class and leverages this framework to process new "SimEvents" added for the scheduling framework. This allows the scheduler to easily handle task arrivals and completions and also log metrics using the new "SchedulerMetrics" class; also added during the project, which is discussed in later sections.

All scheduling logic resides in this class/sub-classes so it essentially acts as the orchestrator for running experiments. The class includes a list of type "EdgeServer", a reference to a cloud server, and a queue of type "PriorityTask". The class exposes methods to allow any class implementing it to add tasks to any one of the edge servers or to the cloud.

The AbstractScheduler itself cannot be instantiated as it is an abstract class with abstract methods, namely handleTaskArrival(), handleTaskCompletion() and schedule(). A user of the framework who wishes to implement their own algorithm only needs to extend the new AbstractScheduler class and implement the three abstract methods to have a fully functioning and measured scheduler built into the simulator. Both the new PATS algorithm along with a FCFS baseline algorithm are implemented in this way to demonstrate the new scheduling layer's flexibility.

4.5 Edge-state Snapshot

As discussed, the new EdgeServer entity which was introduced as part of the framework utilises CloudSim's event-driven architecture for handling task arrivals from the scheduler and reporting task completions back to it. As a result, during a single cycle of the scheduler's schedule method, tasks dispatched at a given time instance do not arrive at servers instantaneously due to the asynchronous nature of the event-driven framework. To address this, an EdgeState class which acts as a lightweight snapshot of each server's current state was also created. This allows the scheduler to make informed placement decisions based on each server's estimated capacity and utilisation within the same scheduling cycle, even when previously dispatched tasks have not yet physically arrived at their destination server.

4.6 Scheduler Metrics

The "SchedulerMetrics" class is responsible for capturing all performance metrics produced during a simulation run. It records task-level data including submission counts, completion times, cloud offload counts, and preemption events, and aggregates these across both priority levels. Although not all metrics are relevant in this study, they were included to allow future researchers to study all key aspects regarding scheduler performance.

A central metric of this study is Quality of Experience (QoE). Although much of the recent literature proposes various metrics for quantifying user experience with an application, including packet loss and network jitter, latency remains the most commonly used measure and is the metric adopted in this study. In this project, latency is calculated per-task as a function of the ratio between actual completion time and the task's latency requirement [29]. Below is the equation utilised in this project for calculating per task QoE:

$$\text{QoE}(\text{task}) = \max\left(0,\ 1 - \gamma_p \cdot \frac{\max(0,\ T_{\text{response}} - L)}{L}\right) \tag{4.3}$$

where $T_{\text{response}} = t_{\text{completion}} - t_{\text{submission}}$ is the total response time of the task, L is the task's latency requirement in milliseconds, and γ_p is the priority-sensitive degradation factor which controls how aggressively QoE decreases for a given delay. A value of $\gamma_p = 2.0$ is used for high-priority tasks and $\gamma_p = 0.5$ for low-priority tasks, chosen so that high-priority tasks lose all QoE when response time exceeds latency requirement by 50% and low-priority task QoE degrade over three times their latency requirement. This results in high-priority tasks being penalised more aggressively for the same relative delay, reflecting stricter service expectations. Below is the graph displaying the decrease of QoE for a task with a 50ms latency requirement, for both high and low-priority tasks.

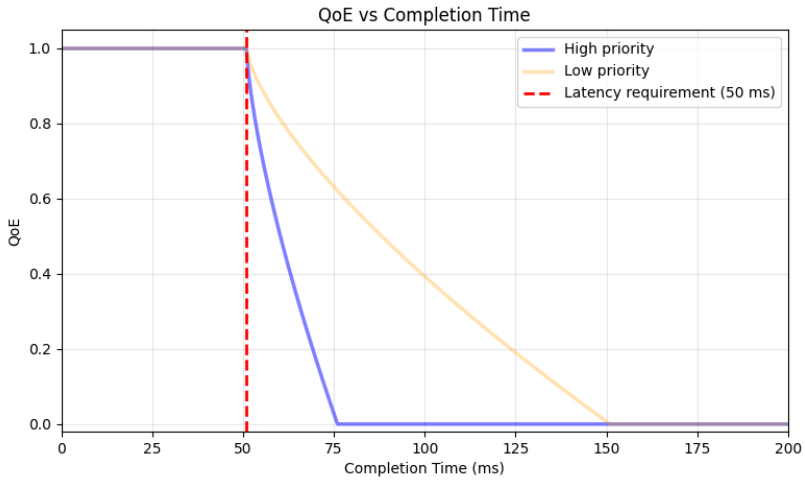


Figure 4.2: High vs. Low-priority QoE Degradation

The class tracks per-priority QoE and is central to this project as it allows for study of high-priority QoE, quantifying the improvement of the PATS algorithm over non-priority aware algorithms.

Energy consumption is also tracked among all active servers and allows users to analyse in greater detail, the use of energy within MEC environments. Although this study gauges energy efficiency through tracking the total number of active edge servers, future extensions of the framework will be able to obtain fine grained tracking of energy consumption through the already integrated functionality within the metrics class.

4.7 Scheduling Algorithms

As one of the contributions of this work being the new scheduling layer within iFogSim2, a baseline algorithm needed implementation, not only to serve as comparison to the PATS algorithm, but also for the purpose to display the new scheduling layers capabilities for implementing distinct scheduling algorithms.

The FCFS algorithm is a relatively straightforward approach to scheduling within MEC environments. It deals with tasks in order of arrival, showing no awareness of task priority or requirements. The algorithm follows a distinct order in which it considers edge servers, also showing no awareness to current system state. The cloud is also available to the scheduler, in situations when tasks cannot be placed onto any local edge servers.

Algorithm 1 FCFS Task Scheduling

```
edgeStateMap ← collectEdgeStateMap()
for all task ∈ getTaskQueue() (in arrival order) do
    state ← nextServer(edgeStateMap)
    if state ≠ null then
        assignTaskToEdgeServer(task, state)
        Update edgeStateMap with new assignment
    else
        assignTaskToCloud(task)
    end if
end for
removeActiveTasksFromQueue()
```

4.8 PATS Algorithm

The central issue with the FCFS algorithm discussed is its inability to distinguish between tasks of different priorities, thus not handling heterogeneous users well. Another issue arising is that the naive approach does not take system state into account when making scheduling decisions. It instead adopts a simple round-robin approach to assigning tasks to edge servers and only making use of the cloud when edge server loads exceed some threshold. Therefore, this presents an opportunity to create an improved, novel approach to scheduling within MEC environments.

The "Priority-Aware Task Scheduling" algorithm, or PATS algorithm for short, was created in an attempt to solve these issues. The algorithm is a dynamic, priority-aware, and system-aware approach to handling heterogeneous tasks in MEC environments. This algorithm makes use of novel functionality such as priority-based task migration along with dynamic cloud offloading, both of which are discussed in detail in the remainder of this chapter.

4.8.1 Priority Scoring Function

Rather than relying on arrival order, the scheduler computes a composite score for each task based on three dimensions. This is the backbone of the PATS algorithm and determines the order in which waiting tasks are considered for scheduling. Following, is the scoring function and a brief explanation of the terms.

$$Score(\text{task}) = A \cdot \text{priorityWeight} + B \cdot \text{waitingTime} + C \cdot \frac{1}{\max(\text{timeToLive},\ 1.0)} \tag{4.4}$$

- $(A \cdot \text{priorityWeight})$: This term encodes the intrinsic importance of the task. High-priority tasks, such as those associated with health monitoring or autonomous vehicle processing, receive a larger priority weight, ensuring they receive a higher priority score thus, placing them higher in the queue.
- $(B \cdot \text{waitingTime})$: This term reflects the total waiting time of a task. The purpose of this term is so that as tasks waiting times increase, naturally their priority should also proportionally increase, to avoid task starvation.
- $(C \cdot \frac{1}{\max(\text{timeToLive},\ 1.0)})$: This term reflects the latency requirement of a task. Tasks with tighter latency deadlines receive a higher urgency score, allowing the scheduler to recognise that work is near to missing its deadline, so that it can be processed.

Each value is normalised to the range of 0 and 1 so that higher values do not skew priority scores. The coefficients A, B, and C are configurable, which will allow administrators to tune the algorithm's behaviour to suit different deployment scenarios. For example, a user may choose to increase C relative to A to create a more deadline-driven scheduler, which could lead to situations where lower-priority tasks with tighter latency requirements take precedence over higher-priority tasks. However, for this study, priority-awareness is the key factor in scheduling decisions, therefore the following assignment is used, which puts the highest weight on the priorityWeight:

$$A = 2, \quad B = 1, \quad C = 1 \tag{4.5}$$

4.8.2 Scheduling Logic

After computing priority scores, tasks waiting in the queue are sorted in descending order by score. Each task then goes through a defined sequence in which the scheduler attempts to find the best placement option given the current system state.

Algorithm 2 Priority-Aware Task Scheduling

```
edgeStateMap ← collectEdgeStateMap()
waitingTasks ← copy of getTaskQueue()
systemLoad ← collectSystemLoad()
Sort waitingTasks in descending order by priority score
for all task ∈ waitingTasks do
    bestState ← findBestServer(task, edgeStateMap)
    if bestState ≠ null then
        assignTaskToEdgeServer(task, bestState)
        Update edgeStateMap with new assignment
    else if isCloudFeasible(task, edgeStateMap) then
        assignTaskToCloud(task)
    else
        resolveConflict(task, edgeStateMap)
    end if
end for
removeActiveTasksFromQueue()
```

The above pseudocode is the core of the PATS algorithm, and is what is implemented in the `schedule()` method as part of the abstract scheduling framework. The scoring function $S(\text{task})$ determines the processing order, after which each task is evaluated through a tiered placement strategy. The scheduler first attempts to find a suitable edge server via `findBestServer()`, falling back to cloud offloading if edge placement is not feasible, and finally invoking `resolveConflict()` as a last resort before simply just letting the task remains in the queue for the next scheduling cycle. Each of these components is discussed in detail in the following subsections.

4.8.3 Find Best Edgeserver

Algorithm 3 Find Best Server for High-Priority Task

Require: $task$, $edgeStateMap$
$bestState \leftarrow null$
for all $edgeState \in edgeStateMap$ **do**
  if $edgeState.efficiency < MIN_EFFICIENCY$ **or** $\neg$ *canAllocateRam(edgeState, task.ramReq)* **then**
    continue
  end if
  if $task.estimatedQoE(edgeState.completionTime) < QoE_{max}$ **then**
    continue
  end if
  if $bestState = null$ **or** $edgeState.remWork < bestState.remWork$ **then**
    $bestState \leftarrow edgeState$
  end if
end for
return $bestState$

Algorithm 4 Find Best Server for Low-Priority Task

Require: $task$, $edgeStateMap$
$bestState \leftarrow null$
$fallbackState \leftarrow null$
for all $edgeState \in edgeStateMap$ **do**
  if $edgeState.efficiency < MIN_EFFICIENCY$ **or** $\neg$ *canAllocateRam(edgeState, task.ramReq)* **then**
    continue
  end if
  if *wouldImpactHighQoE(edgeState)* **then**
    continue
  end if
  if $edgeState.numHighPriorityTasks = 0$ **then**
    if $bestState = null$ **or** $edgeState.remWork < bestState.remWork$ **then**
      $bestState \leftarrow edgeState$
    end if
  else
    if $fallbackState = null$ **or** $edgeState.remWork < fallbackState.remWork$ **then**
      $fallbackState \leftarrow edgeState$
    end if
  end if
end for
return $bestState$ if $\neq null$, else $fallbackState$

A key feature of the findBestServer() method is that it takes a tiered approach for attempting to assign a specific task to an edge server, which depends on the priority of the incoming task. This allows the scheduler to essentially provide better levels of service for the highest priority of tasks. For all priority levels, there are some basic feasibility checks which must be fulfilled before an edge server is considered. The server must have a sufficient amount of available RAM, so that the incoming task can be processed, and the server must also be operating above the minimum efficiency threshold, which can be set as part of the edge server definition. By meeting the minimum threshold of performance, it ensures that edge servers are providing tasks meaningful amounts of MIPS, as degradation of efficiency can lengthen processing time for tasks on the server.

Once the initial resource checks have been conducted, the next step of the scheduler is to evaluate the priority tag associated with the task. For high-priority tasks, there must exist at least one server that can complete the task within its latency requirement. Determination of whether an edge server can meet this requirement is made by examining the number of tasks currently executing on the server, the amount of MIPS available, and the efficiency loss resulting from the addition of another task. It must be verified that the estimated time required to finish the task, after considering these conditions, would meet the latency requirement for the high-priority task. In evaluating all possible edge servers that could host the task, the scheduler chooses the server with the minimum amount of total remaining work among those that meet the feasibility requirements. This approach results in high-priority tasks being placed on the server that would complete them fastest, thus balancing the workload among edge servers.

For low-priority tasks, a stricter set of requirements must be met in order to place such a task on an edge server. While the server must still meet both the efficiency and RAM requirements, the scheduler must also determine whether the placement of the low-priority task on a given server would lower the estimated QoE of any high-priority task currently executing on the server. Should the placement of the low-priority task on the server cause a high-priority task to have a QoE below the maximum QoE, the server is no longer considered for task placement. The scheduler also prefers to use servers that do not have any high-priority tasks executing on them however, the scheduler can fall back to using servers that contain a mix of tasks, should no server with only low-priority tasks be identified.

4.8.4 Resolve Conflict

Algorithm 5 Resolve High-Priority Conflict

Require: $task$, $edgeStates$

$targets \leftarrow$ *findLowCostMigrationTargets(edgeStates, task)* {Greedily evict low-priority tasks from least-loaded server until QoE_{max} and RAM satisfied}
if $targets \neq \emptyset$ **then**
  for all $target \in targets$ **do**
    preemptAndReassign(target)
  end for
  assignTaskToEdgeServer(task, targets.sourceServer)
  return
end if
if no server found **then**
  if $QoE_{cloud} \geq QoE_{bestEdge}$ **then**
    assignTaskToCloud(task)
  else
    Task remains in queue for next cycle
  end if
end if

Algorithm 6 Resolve Low-Priority Conflict

Require: $task$, $edgeStates$
1: **if** $systemLoad =$ HEAVY **then**
2: $threshold \leftarrow 0.75 \cdot QoE_{bestEdge}$
3: **else**
4: $threshold \leftarrow QoE_{bestEdge}$
5: **end if**
6: **if** $QoE_{cloud} \geq threshold$ **then**
7: *assignTaskToCloud(task)*
8: **else**
9: Task remains in queue for next cycle
10: **end if**

The resolveConflict() function is the last check before a task simply remains in the task queue buffer for the next cycle. Like findBestServer(), this algorithm takes a tiered approach, handling high-priority and low-priority tasks differently. A key difference between the handling of the two priority classes is that for high-priority tasks, the scheduler will consider preemption and migration. It first attempts to identify low-priority tasks currently running on edge servers that can be preempted to free resources for the high-priority task. The scheduler favours the least-loaded server, minimising the number of preemptions required and thereby reducing disruption. Among potential candidate tasks to migrate, it prefers preempting those with the least work completed which attempts to avoid disrupting low-priority tasks that are near completion and at risk of missing their latency deadlines. Once candidates are identified, they are preempted and a migration is attempted to reassign them to another edge server and then the incoming task which triggered the preemption gets placed onto the edge server with newly freed resources. If another edge server is not currently available for the preempted task, it will reenter the task queue but importantly, tasks which have been preempted will naturally receive higher priority scores than newly submitted tasks. This is because their earlier submission times give them lower time-to-live values, preventing the starvation of work that was previously in progress.

In the case in which no suitable targets exist, before the task simply remains in the queue for the next cycle, the algorithm will consider the possibility of offloading to the cloud if latency requirements permit. It compares the expected QoE of offloading the task to the cloud against the expected QoE of keeping it on the edge. The cloud QoE is estimated by factoring in the round-trip time for the task to be sent to the cloud, processed, and returned. The edge QoE is estimated by examining the current workload on edge servers, the task position in the queue, and how long it would take the task to be executed. If the cloud offers a better QoE than waiting, the task is offloaded, otherwise, it remains in the queue for the next scheduling cycle.

In contrast, the handling of low-priority tasks is not as complex as handling high-priority tasks, due to migration not being considered. The only decision is whether to offload the task to the cloud or keep it on the edge. The scheduler takes an adaptive approach to deciding the criteria of low-priority task offloading, depending on the overall system load at the current time. The overall scheduler load is defined to be normal when 75% of monitored edge servers are reported to be in light or idle load. However, when at least 75% are reported to be under heavy load, defined in Section 4.3, the overall system load is considered to be heavy. Under normal system load, the expected QoE resulting from cloud offloading must be as good as the expected QoE if the task were to stay on the edge. However, under heavy system load, the acceptance threshold is reduced and if the cloud QoE is at least some percentage of the expected edge QoE, the task will be offloaded. This is done in the above pseudocode by the threshold variable, which drops to 75% of the best edge QoE during heavy load. The cloud QoE is then compared against this threshold and if it meets the requirement, the task is offloaded, else, it remains in the queue. The rationale behind this decision is that under heavy system load, it is of more interest to offload more low-priority

tasks rather than keeping them on the edge, which could possibly further saturate edge resources. By doing so, more high-priority tasks can be processed locally with less disruption and in general, allows the scheduler to respond to system pressure effectively by offloading lower-importance work to the cloud, freeing edge capacity for higher-priority or newly arriving tasks. The QoE threshold for low-priority cloud offloading is a configurable parameter within the scheduler and for the purposes of this study, 0.75 was selected. This is high enough to prevent unnecessary offloading under normal conditions, but low enough to trigger offloading before edge QoE degrades significantly.

4.9 Time Complexity Analysis of PATS

Following is a formal analysis of the time complexitiy of the proposed PATS algorithm. Let n denote the number of tasks currently in the waiting queue at the start of a scheduling cycle, m the number of edge servers configured in the MEC environment, and k the maximum number of concurrently running tasks on any single edge server.

4.9.1 Scoring, Sorting and Best-Server Selection

At the beginning of every invocation of the schedule() method, a priority score is computed for each task in the queue in constant time, contributing $O(n)$ overall for each of the n tasks in the queue. The queue is then sorted in descending order of priority score which results in an addition of $O(n \log n)$.

After sorting, the scheduler attempts to find the best edge server for each task via findBestServer(). For high-priority tasks, this is a linear scan over all edge servers with constant-time feasibility checks, resulting in $O(m)$ per task. However, for low-priority tasks, the scheduler additionally checks whether placement would impact the QoE of any high-priority tasks currently running on the candidate server, contributing $O(k)$ per server and therefore $O(m \cdot k)$ per task. Across all n tasks, the worst-case cost of best-server selection is $O(n \cdot m \cdot k)$.

4.9.2 Conflict Resolution

When no feasible edge server can be found for a high-priority task, the scheduler invokes resolveConflict(), which in turn calls findLowCostMigrationTargets() to identify low-priority preemption targets. For each candidate server, the active low-priority tasks are sorted by remaining work in $O(k \log k)$, after which preemption targets are greedily selected until the RAM and QoE requirements of the incoming task are satisfied. In the worst case this contributes $O(m \cdot k \log k)$ per invocation, or $O(n \cdot m \cdot k \log k)$ across a full scheduling cycle in the pathological case where every high-priority task in the queue triggers conflict resolution. If no suitable targets are found, the fallback path which is utilised for both low and high-priority tasks compares the expected QoE of cloud offloading against edge execution. To efficiently calculate the total amount of work remaining ahead for a specific task a prefix-sum array is used. This array is built at the start of the cycle and so adds no additional asymptotic cost, providing a constant, $O(1)$ approach for quantifying how much work remains ahead in the queue of a specific task.

4.9.3 Overall Complexity and Comparison with FCFS

Combining the terms above, the per-cycle cost of PATS is the sum of scoring, sorting, best-server selection, and conflict resolution. The scoring and sorting are each linear or near-linear in terms of n, while best-server selection and conflict resolution scale with $n \cdot m \cdot k$ and $n \cdot m \cdot k \log k$ respectively. The last term dominates, yielding a worst-case per-cycle complexity of:

$$O(n \cdot m \cdot k \log k) \tag{4.6}$$

In realistic MEC deployments, both m and k are small and bounded - edge clusters typically contain only a handful of servers, and the minimum efficiency threshold defined caps the number of concurrent tasks on any single server. Under these conditions the $m \cdot k \log k$ factor behaves as a small constant, and the per-cycle cost scales close to linearly with n. By comparison, although FCFS follows a round-robin approach for edge selection, `nextServer()` must still linearly probe up to m servers per task to find one with sufficient capacity to host the task, yielding a worst-case time complexity of $O(n \cdot m)$ per-cycle. PATS therefore incurs only a modest overhead in typical conditions in exchange for its priority-awareness, intelligent placement, and migration capabilities.

4.10 System Model

Edge-Cloud Topology and Priority Label Trust

The simulations conducted in this study model a simplified MEC topology in which a small number of homogeneous edge servers are configured at the network edge alongside a single remote back-up cloud server. All edge servers share identical specifications in terms of processing capacity, available RAM, and network latency to reflect a small, independent edge cluster. For the purpose of this study, we assume that edge servers and users are in close proximity to one another, leading to negligible latency. Thus, the network latency is set to 0 for edge servers in the following simulations, but utilising the cloud server incurs the cost of a round-trip latency penalty [30], however provides substantially greater computational power than the local edge cluster. Another assumption of this model is that the priority label attached to each incoming task is trusted by the scheduler. This avoids the need to impose rate-limiting constraints or SLA agreements for this study, so that the issue of scheduling policy can be taken in isolation. Assumptions made for this study are revisited in Section 6.2. Below is a diagram to illustrate how the topology is configured:

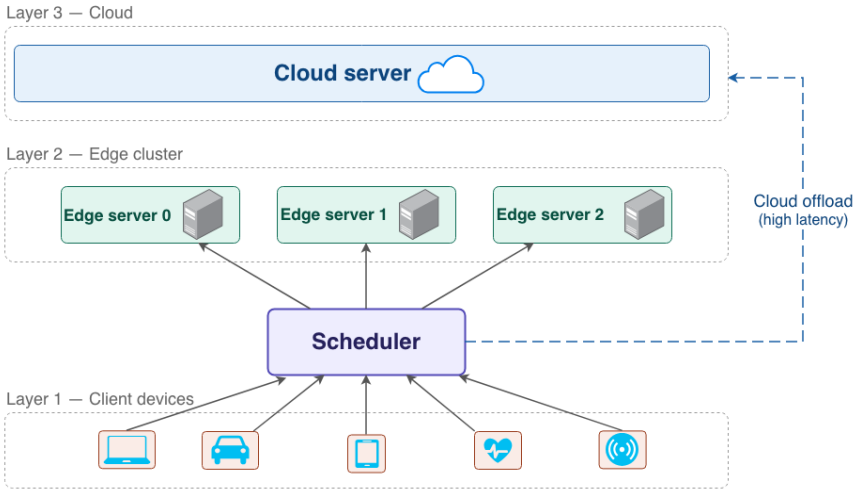


Figure 4.3: Three-Tier Computing Architecture with Centralised Scheduler

Server Efficiency

As stated earlier in the definition of the purpose-built EdgeServer entity, server efficiency is calculated as a function of the number of concurrently executing tasks. Under this model, active tasks receive a fair-share portion of the available MIPS capacity, which is further scaled by an efficiency factor to account for context-switching overhead associated with concurrent execution. Initial experiments were carried out to analyse the impact of increasing task load on edge server performance. System performance was shown to degrade noticeably as server efficiency decreased as a result of context-switch overhead. However, it was found that maintaining a minimum server efficiency of approximately 80% allowed concurrent execution to achieve acceptable performance levels, while lower efficiency values resulted in significant degradation in task completion times. Therefore, in the system model used in this study, edge servers are required to operate at a minimum efficiency threshold of 80% which is applied across both the baseline FCFS scheduling simulations and the proposed PATS scheduling simulations.

Migration Penalty

The migration delay for a task moving to another server is modelled as:

$$T_{migration} = \frac{RAM_{task}}{BW_{edge}} + \delta \tag{4.7}$$

where RAM_{task} task is the memory footprint of the migrating task (MB), BW_{edge} is the inter-edge server bandwidth (MB/ms), and δ is a fixed overhead representing context teardown. This formulation follows an idealised abstraction of the work by Machen et al. [31] which states the linear relationship between migration time, data size, and bandwidth in MEC container migration scenarios, while abstracting away some of the lower-level system overheads presented in their work. For the edge cluster modelled in this study, an inter-edge bandwidth of 1 GB/s and a fixed overhead of 5ms are assumed, reflecting closely located edge servers within a MEC environment. A small initialisation overhead also applies to the incoming task on the freed server, however, for simplicity, this is assumed to be negligible relative to the task's execution time.

Abstractions

A key generalisation of this model is that the devices and applications are abstracted away, and the system does not simulate individual users, mobility patterns, or application-level behaviour, like other integrations within the iFogSim2 framework. Instead, the workload entering the scheduler is represented as a stream of small tasks, each with computational requirements, memory demands, a latency requirement, and a most importantly, a priority label. These tasks are not intended to model complete application workloads but instead, they model small, discrete units of work that could form part of a larger service pipeline. For example, a single task may represent an inference call for a cardiac monitoring algorithm [32] or one frame of an AR rendering pipeline [30].

This abstraction allows the study to focus more on the scheduling problem and evaluating how different algorithms allocate constrained edge and cloud resources across heterogeneous tasks, without considering higher level details related to user behaviour or device heterogeneity. While in this study, configurations comprise of homogenous edge servers and two types of tasks, the abstract scheduling framework is not limited solely to this configuration. For future studies which use the new scheduling framework, it is possible to look at even more heterogeneous environments in which edge servers may have different specifications or in which tasks of similar priority label, may have different compute, memory and latency requirements.

Chapter 5: Simulation & Results Analysis

5.1 Verification Testing

The purpose of this section is to ensure the newly implemented PATS scheduler makes correct scheduling decisions, as per its design and implementation. Ensuring the scheduler exhibits correct behaviour is crucial, therefore, several tests will be run under specific headings of the schedulers' design to examine its decisions.

Each set of simulations will be configured differently to put the scheduler under scenarios in which it must make the intended decisions of the specific test case. Certain characteristics of the MEC environment will hold true for each test case unless otherwise stated, such as edge and cloud server MIPS, RAM and latency, along with task MIPS, RAM and latency requirements. For simplicity, it is assumed that all edge servers used in these scenarios have similar specifications. Below is a table outlining the baseline configuration:

Table 5.1: Verification Test Configuration

Category	Parameter	Value
Edge Server	Processing Capacity (MIPS)	2,000
	RAM (MB)	200
	Latency (ms)	0
Cloud Server	Processing Capacity (MIPS)	44,800
	RAM (GB)	1,024
	Latency (ms)	75
Tasks	Task MIPS Requirement	20
	Task RAM Requirement (MB)	2

It is also relevant to note for tests in which latency requirements are not factored in scheduling decisions, that it is assumed that tasks have relaxed latency requirements, allowing the scheduler to make isolated decisions based on other factors. This will be true for any test cases which don't explicitly mention task latency requirement values.

5.1.1 Priority-Aware Task Ordering

The purpose of this first test is to show a simple example of the scheduler executing its main objective; scheduling tasks labelled as high-priority before low-priority tasks. To demonstrate this scenario, one edge server will be initialised. Edge server RAM will be set to ensure that it can only host one task at a time, by lowering its capacity to match the RAM requirement of the incoming tasks. Two tasks will be submitted to the scheduler at t=0, one labelled high-priority, and one labelled low-priority, with arbitrary latency requirements as they are not factored into the scheduling policy for this test. Although both tasks have identical submission time, they are added sequentially to the task list in the simulation code, which introduces a deterministic ordering in which the low-priority task is added first. This ensures that any difference in scheduling outcome

is due to the scheduler's priority handling rather than nondeterministic task arrival.

The expected behaviour is that when both tasks arrive, they are sorted by priority score, and high-priority task is placed onto the single edge server. It should then complete before the low-priority task is then placed onto the same edge server.

Results & Discussion

```
Starting CloudSim version 7.0
0.0: pats-scheduler is starting...
Entities started.
0.0: edge-0: Task #high-0 arrived (20.0 MI remaining)
10.0: edge-0: Task #high-0 completed.
10.0: edge-0: Task #low-0 arrived (20.0 MI remaining)
20.0: edge-0: Task #low-0 completed.
```

Figure 5.1: PATS output

```
Starting CloudSim version 7.0
0.0: fcfs-scheduler is starting...
Entities started.
0.0: edge-0: Task #low-0 arrived (20.0 MI remaining)
10.0: edge-0: Task #low-0 completed.
10.0: edge-0: Task #high-0 arrived (20.0 MI remaining)
20.0: edge-0: Task #high-0 completed.
```

Figure 5.2: FCFS output

The above output logs show the order in which events occur, and as shown in the snippet, the PATS scheduler is correctly placing the high-priority task before the low-priority task which aligns with the behaviour expected, demonstrating that the scheduler is priority-aware in its decision-making. Despite the deterministic ordering of tasks in the submitted task list, the high-priority task is still considered ahead of the low-priority task, even though the low-priority task was intentionally added to the task list before the high-priority task to highlight this behaviour. By observing the same scenario, but with the FCFS scheduler, the low-priority task gets placed before the high-priority task, displaying its lack of awareness to priority labels.

5.1.2 Load-Aware Edge Server Selection

This test verifies that the PATS scheduler assigns newly submitted tasks to the least loaded edge server, rather than using a round-robin approach, as observed in the FCFS approach. In this scenario, the edge topology now includes two edge servers. At the start of the simulation, two high-priority tasks are running on edge-0, and one high-priority task is running on edge-1. At t=5, a new high-priority task arrives into the system to be placed onto an edge server.

The expected behaviour is that PATS places the newly submitted task onto the least loaded edge server, which in this case is edge-1, allowing all tasks to complete with minimal latency.

Results & Discussion

```
Starting CloudSim version 7.0
0.0: pats-scheduler is starting...
Entities started.
0.0: edge-0: Task #load-edge0-a arrived (20.0 MI remaining)
0.0: edge-0: Task #load-edge0-b arrived (20.0 MI remaining)
0.0: edge-1: Task #load-edge1-a arrived (20.0 MI remaining)
5.0: edge-1: Task #load-test-task arrived (20.0 MI remaining)
```

Figure 5.3: PATS output

```
Starting CloudSim version 7.0
0.0: fcfs-scheduler is starting...
Entities started.
0.0: edge-0: Task #load-edge0-a arrived (20.0 MI remaining)
0.0: edge-0: Task #load-edge0-b arrived (20.0 MI remaining)
0.0: edge-1: Task #load-edge1-a arrived (20.0 MI remaining)
5.0: edge-0: Task #load-test-task arrived (20.0 MI remaining)
```

Figure 5.4: FCFS output

As shown in Figure 5.3, displaying the output of the PATS simulation, the new task gets correctly placed onto edge-1, which is the least loaded of the two. Looking at Figure 5.4, we see that FCFS follows a Round-Robin approach to assigning tasks to edge servers, resulting in three tasks being placed onto edge-0. This test displays PATS ability to conduct load balancing within the

MEC environment, which can have a major impact on overall system performance under real-world scenarios.

5.1.3 Default Isolation of High-Priority Tasks

This test verifies that the scheduler protects high-priority tasks by attempting to avoid placing low-priority tasks onto edge servers with high-priority tasks, even when doing so would not affect any high-priority task QoE. The edge topology consists of two edge servers, one running a high-priority task and the other running a low-priority task. At t=5, a new low-priority task is submitted to the scheduler for execution.

The expected behaviour in this scenario is that PATS will prefer to place the new task onto the server not containing the high-priority task. Although the task latencies are configured such that the placement of any task onto edge-0 would not affect the high-priority task, PATS will still defer to avoiding the edge server hosting high-priority task entirely.

Results & Discussion

```
Starting CloudSim version 7.0
0.0: pats-scheduler is starting...
Entities started.
0.0: edge-0: Task #high-0 arrived (20.0 MI remaining)
0.0: edge-1: Task #low-0 arrived (20.0 MI remaining)
5.0: edge-1: Task #low-1 arrived (20.0 MI remaining)
```

Figure 5.5: PATS output

```
Starting CloudSim version 7.0
0.0: fcfs-scheduler is starting...
Entities started.
0.0: edge-0: Task #high-0 arrived (20.0 MI remaining)
0.0: edge-1: Task #low-0 arrived (20.0 MI remaining)
5.0: edge-0: Task #low-1 arrived (20.0 MI remaining)
```

Figure 5.6: FCFS output

As shown in Figure 5.5, PATS places the newly submitted task onto the edge server containing the low-priority task, as expected, protecting the high-priority task. Looking at Figure 5.6, it is observed that FCFS, follows its predefined cycle for selecting edge servers, leading to the placement of the low-priority task onto edge-0, thus impacting the high-priority task latency.

5.1.4 Permitted Co-location When QoE Is Preserved

The following test verifies that while it is not preferred, PATS can place a low-priority task onto an edge server with an already running high-priority task. This is to prevent unnecessary avoidance of co-locating tasks simply due to priority labels, and instead allowing tasks to be placed together when options are restricted, provided task QoE is not affected. The setup for the following test includes just one edge server with an executing high-priority task with a relaxed latency requirement of 100ms. At t=5, a low-priority task arrives into the network.

The expected behaviour is that the low-priority task is subsequently placed onto edge-0, along with the high-priority task. In this scenario, this is permitted due to the high-priority tasks relaxed latency requirement, which would not be affected by the low-priority task.

Results & Discussion

```
Starting CloudSim version 7.0
0.0: pats-scheduler is starting...
Entities started.
0.0: edge-0: Task #high-0 arrived (20.0 MI remaining)
5.0: edge-0: Task #low-1 arrived (20.0 MI remaining)
```

Figure 5.7: PATS output

As displayed in Figure 5.7, the task is successfully placed onto edge-0, which is already hosting a high-priority task. This is the expected outcome given that the high-priority task has a latency requirement of 100ms and only has 20MI of work, allowing it to complete well within 100ms from its submit time.

5.1.5 Rejected Co-location Under QoE Risk

The purpose of this test is to show the opposite of the previous test, that is, PATS will not place a low-priority task onto edge server if it will impact the QoE of any high-priority task running on the edge server. Ensuring the protection of high-priority tasks is a critical ability of the scheduler and is in place to attempt to maintain the highest levels of service for the most important tasks. The configuration of this simulation mirrors the previous setup with the key difference of the high-priority task latency requirement being reduced significantly to 10ms.

It is expected that the low-priority task submitted at t=5 is not placed onto edge-0, as doing so would cause the high-priority task to completely miss its deadline. Instead it remains in the queue allowing the high-priority task to fully complete, and meet its latency requirement.

Results & Discussion

```
Starting CloudSim version 7.0
0.0: pats-scheduler is starting...
Entities started.
0.0: edge-0: Task #high-0 arrived (20.0 MI remaining)
10.0: edge-0: Task #high-0 completed.
10.0: edge-0: Task #low-1 arrived (20.0 MI remaining)
20.0: edge-0: Task #low-1 completed.
```

Figure 5.8: PATS output

```
Starting CloudSim version 7.0
0.0: fcfs-scheduler is starting...
Entities started.
0.0: edge-0: Task #high-0 arrived (20.0 MI remaining)
1.0: edge-0: Task #low-1 arrived (20.0 MI remaining)
21.0: edge-0: Task #high-0 completed.
22.0: edge-0: Task #low-1 completed.
```

Figure 5.9: FCFS output

Figure 5.8 shows that in this instance, PATS does not place the low-priority task onto edge-0, which contains the high-priority task as it recognises the delay that doing so would cause the high-priority task. Figure 5.9 shows that under the exact same simulation, FCFS does in fact place both tasks onto the same edge server, further displaying FCFS lack of awareness for task requirements which causes the high-priority task to miss its deadline.

5.1.6 Preemption to Protect High-Priority QoE

This verification test is one of the more important tests conducted in this section, as it verifies that priority-based preemption takes place when utilising the PATS scheduler. This feature of the scheduler is crucial to maintain high-priority task QoE scores as it allows resources to be made

available for most important tasks. The configuration of this simulation includes two edge servers, each hosting one low-priority task. At t=5, a high-priority task is submitted to the scheduler. In this scenario, the high-priority task requires all available resources of one of the edge servers to meet its latency requirement of 10ms, which would require a preemption.

What is expected is that PATS will be unable to find sufficient free resources to satisfy the high-priority task's latency requirement. This will then lead the scheduler down the "resolve conflict" branch of the algorithm, therefore triggering a preemption of one of the low-priority tasks. The preempted task should then migrate to the other edge server. After the edge server becomes available, the high-priority task should be placed onto it and complete its execution within its latency requirement.

Results & Discussion

```
Starting CloudSim version 7.0
0.0: pats-scheduler is starting...
Entities started.
0.0: edge-0: Task #low-0 arrived (20.0 MI remaining)
0.0: edge-1: Task #low-1 arrived (20.0 MI remaining)
5.0: edge-0: Task #low-0 PREEMPTED (Remaining tasks on server: 0)
5.0: edge-0: Task #high-1 arrived (20.0 MI remaining)
5.0: edge-1: Task #low-0 arrived (10.0 MI remaining)
15.0: edge-0: Task #high-1 completed.
17.0: edge-1: Task #low-1 completed.
17.0: edge-1: Task #low-0 completed.
```

Figure 5.10: PATS output

```
Starting CloudSim version 7.0
0.0: fcfs-scheduler is starting...
Entities started.
0.0: edge-0: Task #low-0 arrived (20.0 MI remaining)
0.0: edge-1: Task #low-1 arrived (20.0 MI remaining)
5.0: edge-0: Task #high-1 arrived (20.0 MI remaining)
10.0: edge-1: Task #low-1 completed.
17.0: edge-0: Task #low-0 completed.
22.0: edge-0: Task #high-1 completed.
```

Figure 5.11: FCFS output

Figure 5.10 displays the successful preemption of task low-0, from edge-0. This allows the high-priority task to utilise all available resources on edge-0, resulting in the task meeting its latency requirement. The preempted task, low-0, is then shown to be migrated to edge-1, incurring the migration cost per its relative size. Looking at Figure 5.11, we see that under the same scenario, FCFS triggers no preemption/migration, resulting in the high-priority task missing its latency requirement of 10ms and significantly degrading its QoE.

5.1.7 Cloud Offload for Latency-Tolerant Low-Priority Tasks

The final verification test has the purpose of confirming that if latency requirements are lenient for a low-priority task, PATS will prefer offloading low-priority tasks to the cloud. This is in an attempt to keep local edge utilisation low, which as a result will allow for any future incoming tasks which may have stricter latency requirements to be executed fully on the edge. The configuration for this test includes just one edge server, and two tasks both with submission time of t=0. One task is labelled high-priority with 50ms latency requirement and the other labelled low-priority, with 100ms of latency requirement. The cloud latency as stated in table 5.1 is just 75ms plus an additional 1ms for execution resulting in a total of 76ms for round-trip time. Both the high and low-priority tasks are submitted at t=0.

The expected outcome is that the high-priority task gets considered and ultimately placed first. Then when considering the low-priority task, PATS recognises that the low-priority latency requirement would be met even if it was offloaded to the cloud. This then allows the scheduler to keep local edge servers free for any future tasks.

Results & Discussion

```
Starting CloudSim version 7.0
0.0: pats-scheduler is starting...
Entities started.
0.0: edge-0: Task #high-0 arrived (20.0 MI remaining)
10.0: edge-0: Task #high-0 completed.
38.0: cloud: Task #low-0 arrived (20.0 MI remaining)
77.0: cloud: Task #low-0 completed.
```

Figure 5.12: PATS output

```
Starting CloudSim version 7.0
0.0: fcfs-scheduler is starting...
Entities started.
0.0: edge-0: Task #high-0 arrived (20.0 MI remaining)
1.0: edge-1: Task #low-0 arrived (20.0 MI remaining)
10.0: edge-0: Task #high-0 completed.
11.0: edge-1: Task #low-0 completed.
```

Figure 5.13: FCFS output

As depicted in Figure 5.12, the high-priority task is correctly placed first, and at t=38 the low-priority task arrives at the cloud, corresponding to approximately half of the cloud round-trip time. At t=77, the low-priority task returns from the cloud, fully completed. As noted, this reflects the 75ms of latency to the cloud plus an additional 1ms for execution, which stems from the cloud's significantly greater computational resources compared to the edge. Figure 5.13 shows that FCFS does not choose to offload the low-priority task to the cloud. While the low-priority task completes in significantly less time with FCFS, this approach becomes problematic when task submission rates increase. In such scenarios, low-priority tasks may occupy edge slots that could otherwise serve higher-priority work which may have tighter latency requirements.

5.2 Performance Testing

The testing in section 5.1 had the main aim of verifying that the key parts of the PATS algorithm perform correctly in testing environments. However, the goal of this section is to put the algorithm under more stress and assess its performance in comparison to the FCFS scheduler.

Testing in this section will first focus on the issue of task volumes. The scheduler will be run across three distinct volumes; low, medium, and high - with task counts of 10, 25, and 50, respectively. For each volume, testing will include a varying percentage of high-priority tasks; being 20%, 40%, 60% and 80%. This will give a proper view of how the optimised scheduler handles modest loads of high-priority tasks to more extreme loads in terms of overall task volume. For each volume and percentage split of high-priority tasks, we will also look at varying the number of edge servers, with values from 1-5. This will allow for insight into how much extra energy use (via additional edge servers) benefits performance

As there is many variables in real world scenarios, for the set of above-mentioned testing, some assumptions will be made in the overall configuration of the modelled MEC environment. These are displayed in the table below and will be held constant throughout the next three sections of testing examining the three distinct task volumes. A single, fixed cloud latency with a value of 100ms is used in the following tests, as suggested by Mao et al. [30] to be a realistic estimate of cloud latency, and one that enables informed scheduling decisions.

Table 5.2: Performance Test Configuration

Category	Parameter	Value
Edge Server	Processing Capacity (MIPS)	2,000
	RAM (MB)	32
	Latency (ms)	0
Cloud Server	Processing Capacity (MIPS)	44,800
	RAM (GB)	1,024
	Latency (ms)	100
Tasks	Task MIPS Requirement	20
	Task RAM Requirement (MB)	2
	High Latency Req. (ms)	35
	Low Latency Req. (ms)	50
	Task Submission Time	0

However, testing from section 5.2.4 on, will investigate how varying other dimensions of the problem, namely adjusting cloud latency and high-priority latency requirements, impact scheduling decisions of the PATS algorithm and compare this with the FCFS approach.

Metrics Graphed

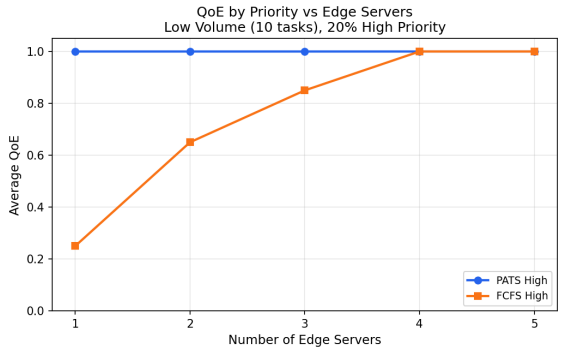
Weighted performance metrics are often utilised to reflect the higher importance of delay-sensitive tasks in MEC systems [33]. As discussed earlier, delay-sensitive, high-priority tasks such as autonomous vehicle navigation require consistently low latency, whereas long-running, low-priority tasks such as file backups are much more delay tolerant. A simple average would be misleading, as it would hide poor performance on critical tasks behind acceptable performance on less important ones. Thus, to gauge overall system performance, high-priority tasks are weighted three times more than low-priority tasks, and the weighted average is displayed, rather than a simple average. While the factor of three is a design choice of this study, it is sufficient to reflect the significantly greater importance of high-priority tasks relative to low-priority ones.

For each set of tests, two graphs are created. The first graph will look exclusively at the QoE of high-priority tasks, providing a focused view of performance for workloads of high importance. The second graph will look at the weighted average of both high and low-priority tasks, giving a comprehensive view on overall system performance.

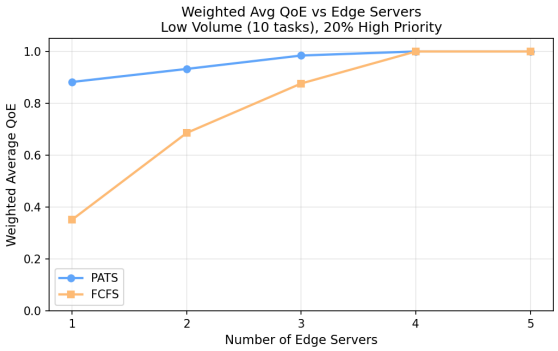
It should be noted that, due to the fact that tasks are submitted in randomised order resulting in runs which are not deterministic, each simulation is run multiple times. Specifically in this study, each simulation was run ten times, and the average was computed, and results displayed are the result of these averages.

5.2.1 Low Volume (10 Tasks)

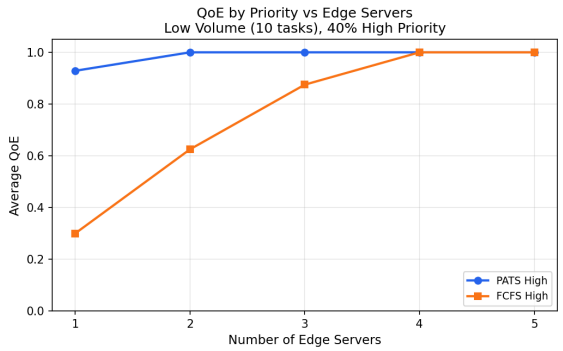
For the first set of tests, we will be considering a relatively small number of tasks submitted at t=0. With this small volume of tasks, the aim will be to see at what point, is QoE maximized, in terms of increasing the number of edge servers. This will provide an initial baseline of how both the PATS and FCFS schedulers perform under minimal load before scaling up to more demanding scenarios.



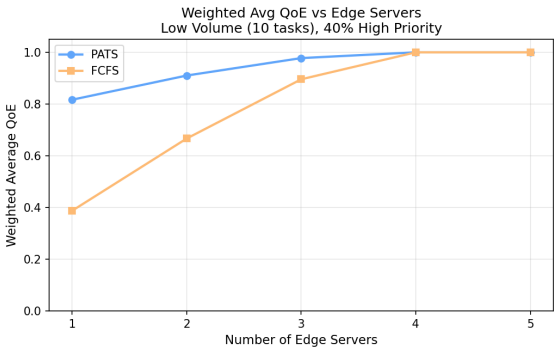
(a) QoE by Priority, 20% High-Priority



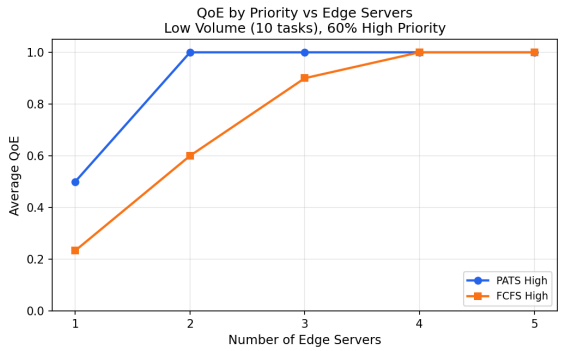
(b) Weighted Avg QoE, 20% High-Priority



(c) QoE by Priority, 40% High-Priority



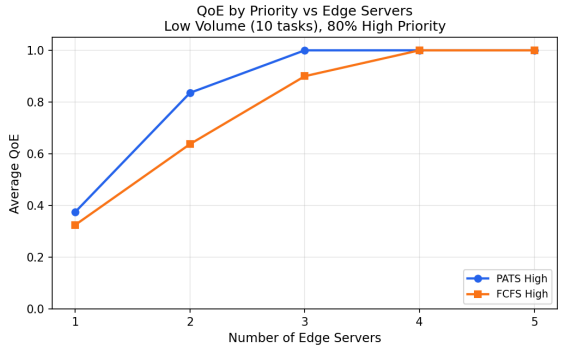
(d) Weighted Avg QoE, 40% High-Priority



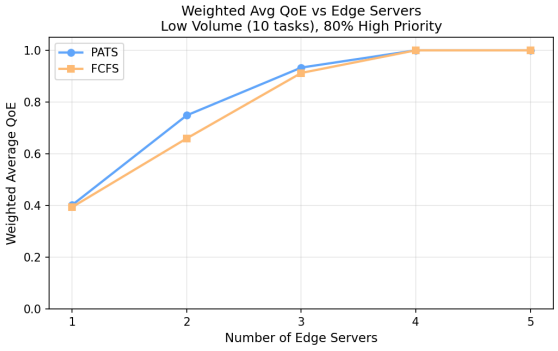
(e) QoE by Priority, 60% High-Priority



(f) Weighted Avg QoE, 60% High-Priority



(g) QoE by Priority, 80% High-Priority



(h) Weighted Avg QoE, 80% High-Priority

Figure 5.14: Low Volume (10 Tasks) Results Across Priority Splits

Discussion of Results

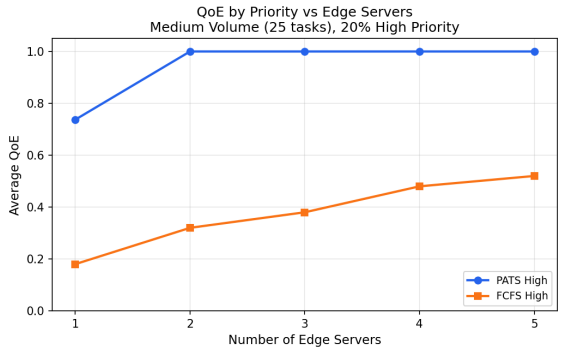
Figure 5.14 shows that PATS consistently achieves higher QoE for high-priority tasks compared to FCFS across all priority splits and edge server configurations. Firstly, by looking at the high-priority QoE in terms of increasing percentage of high-priority tasks, we see that for relatively low percentages, PATS shows a drastic improvement compared to FCFS. Only at from 60% onwards, do we see the results of the PATS approach begin to drop to similar levels as FCFS. However, an interesting point is that the increasing percentages seem to have no real effect on the FCFS scheduler. This can simply be put down to the fact that due to the schedulers' inability to differentiate between priority classes, scheduling policy does not change with the increasing number of high-priority tasks. In contrast, PATS behaviour does change with the increasing number of tasks labelled high-priority. Although a core strength of the PATS scheduler is being able to pick out the most important work out of several incoming tasks, the scheduler begins to see no benefit when it is presented with the majority of work being labelled as high-priority. This is why the scheduler seems to converge closer and closer to the results of FCFS with the increasing number of high-priority tasks.

Examining the results in terms of how performance changes with increasing energy consumption through additional edge servers, what is obvious is that the PATS approach is of greater energy efficiency. With more modest percentages of high-priority tasks, we see far greater performance with the PATS approach with notably fewer edge servers. For example, in Figure 5.14a, we see that while the FCFS scheduler requires four edge servers to completely satisfy all high-priority tasks, PATS only requires one edge server to provide the same level of service. The reasoning behind this is that all available infrastructure is being used to support the processing of high-priority tasks, before seeing to low-priority tasks. Increasing the percentage of high-priority tasks to 40%, maximising QoE only requires a two servers, despite one server providing near maximum QoE levels at around 0.9. This trend of increasing percentage by 20%, and only requiring an additional server each time approximately continues throughout the results obtained. What is notable too when considering both schedulers is that for each extra edge server, we see a far greater improvement in PATS high-priority QoE compared to FCFS. Again, this indicates that intelligent scheduling can reduce reliance on additional edge infrastructure while still maintaining service quality.

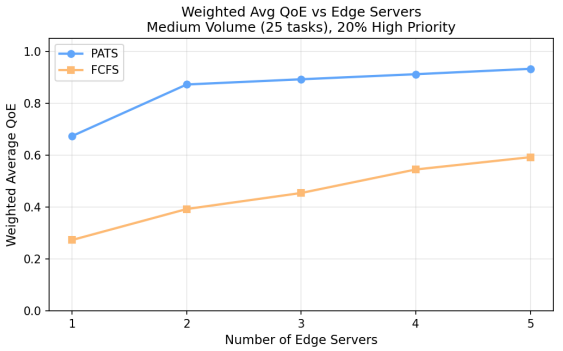
Considering overall system performance for all task types, the weighted average graphs can be observed. What is apparent again is that PATS is still in fact outperforming FCFS. Although there is a small decrease in weighted average QoE compared to high-priority QoE, we see that overall, low-priority work is not being completely sacrificed. The decrease in performance compared to solely the average of high-priority QoE is simply down to the fact that in order to maintain the highest levels of service for the most important work, PATS is forced to provide inferior levels of service to less important work. This is to be expected and is factored into the algorithms design. However, what is clear is that, PATS is providing adequately high levels of service for low-priority tasks, all the while providing the best levels of service to the most important tasks.

5.2.2 Medium Volume (25 Tasks)

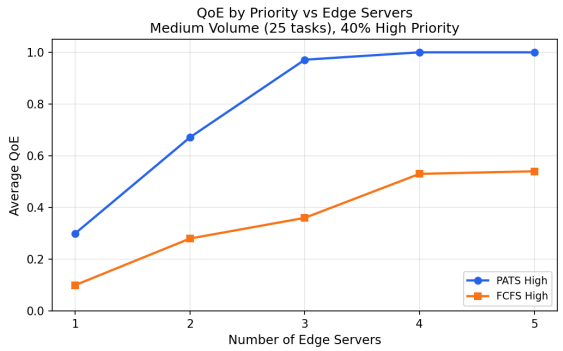
For the next set of tests, we will be considering a more expected volume of tasks submitted at t=0. This set of tests will also be compared with the low task volume to also study if the PATS scheduler follows similar trends for different task volumes.



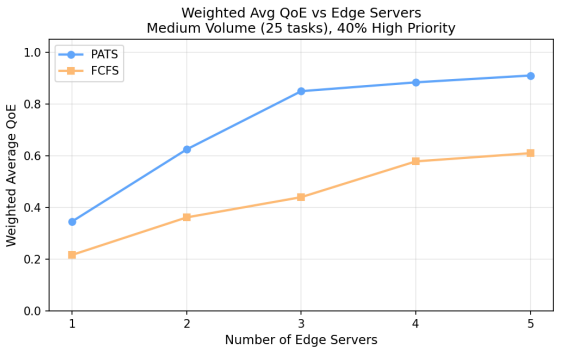
(a) QoE by Priority, 20% High-Priority



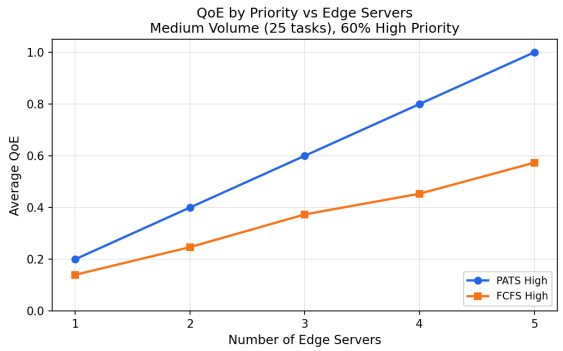
(b) Weighted Avg QoE, 20% High-Priority



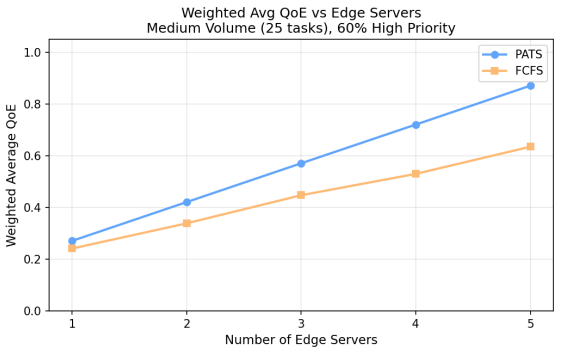
(c) QoE by Priority, 40% High-Priority



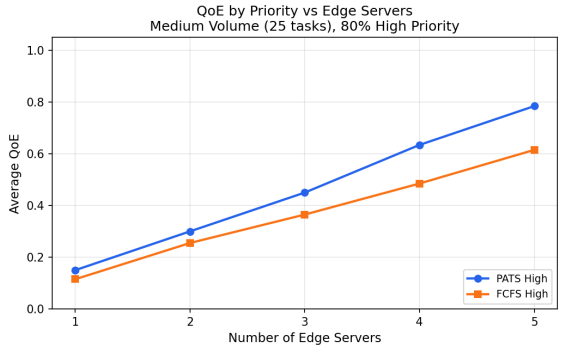
(d) Weighted Avg QoE, 40% High-Priority



(e) QoE by Priority, 60% High-Priority



(f) Weighted Avg QoE, 60% High-Priority



(g) QoE by Priority, 80% High-Priority



(h) Weighted Avg QoE, 80% High-Priority

Figure 5.15: Medium Volume (25 Tasks) Results Across Priority Splits

Discussion of Results

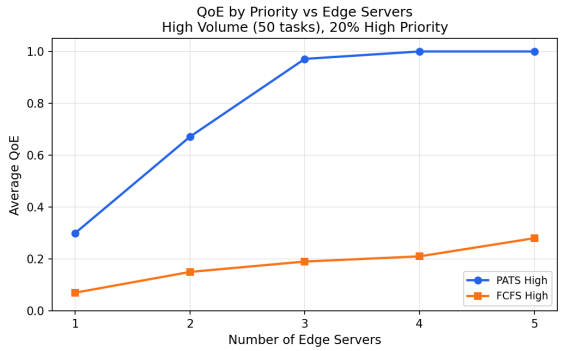
Considering the medium volume of traffic of 25 tasks submitted at t=0, it is clear some trends apparent from the low-volume cases are maintained, though with some features more pronounced. One observation which still holds from the low volume tests is that again, the performance of FCFS through the increasing percentages of high-priority tasks remains somewhat sustained, though; the results are noticeably poorer. For the PATS approach, we see that even with 25 tasks submitted at t=0, for 20% of them labelled high-priority, only two edge servers are required to maximise the QoE of the task class. In Section 5.2.1 each 20% increase in high-priority count required roughly one additional edge server - however at medium volume this scaling required is slightly steeper. This can be observed by comparing the 20% and 40% cases: at 20% high-priority, only two edge servers are needed to fully maximise high-priority QoE, whereas at 40%, three edge servers bring QoE close to, but not quite the maximum. This stems from the extra stress the scheduler is put under, though despite more than doubling the task volume, PATS is still providing more than acceptable levels of service to high-priority tasks

By studying the results in terms of increasing server counts, we are seeing an even greater improvement of PATS over FCFS, compared to the Section 5.2.1. While these benefits are more clearly observed when studying the cases of the high-priority task percentages being no more than 40%, there is still a clear energy consumption improvement over the FCFS approach. While the results obtained in these test case scenarios are similar to what is to be expected after observing the low volume case, the medium volume case further supports the claim that with less energy consumed, superior results are achieved with the PATS algorithm for high-priority work. Again, this is through its intelligent resource utilisation, similar to discussion in Section 5.2.1.

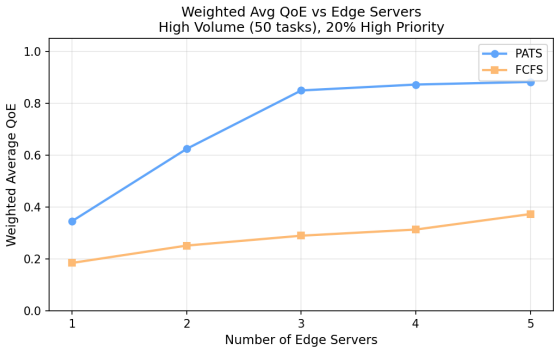
Similar to what was observed from the weighted average graphs acquired in the low-volume test cases, PATS again is showing its advantage over the FCFS algorithm. A point more clearly observed here compared than previously in Section 5.2.1 is how the increasing percentage of high-priority work converges PATS even closer to FCFS. This again follows with the reasoning discussed previously. If the trend follows from increasing task count, it would be expected that when considering the high volume case, the lines would come even closer, compared to the other volume cases.

5.2.3 High Volume (50 Tasks)

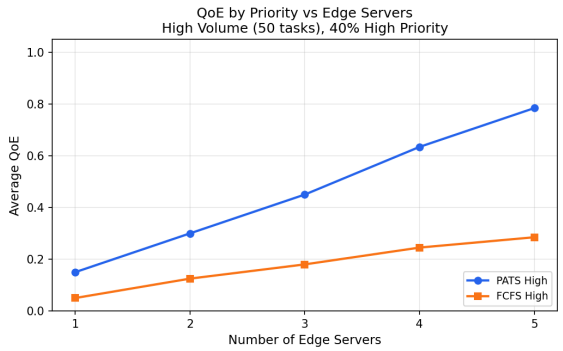
For the final set of tests, a more extreme value for number of tasks submitted will be considered. This set of tests will allow us to observe the PATS algorithms' robustness to high traffic and to also study if the points considered when designed the algorithm offer improved results over FCFS.



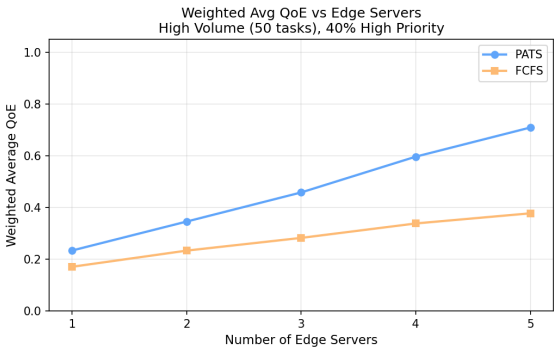
(a) QoE by Priority, 20% High-Priority



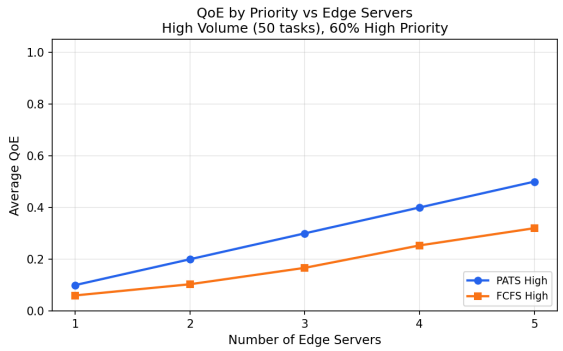
(b) Weighted Avg QoE, 20% High-Priority



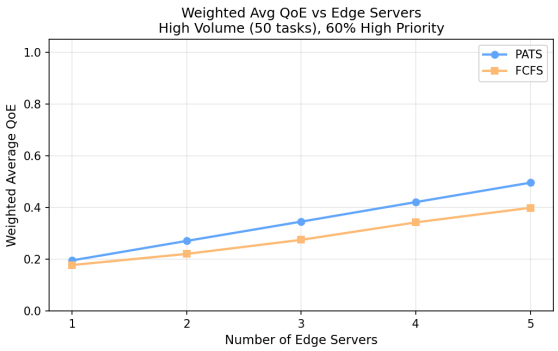
(c) QoE by Priority, 40% High-Priority



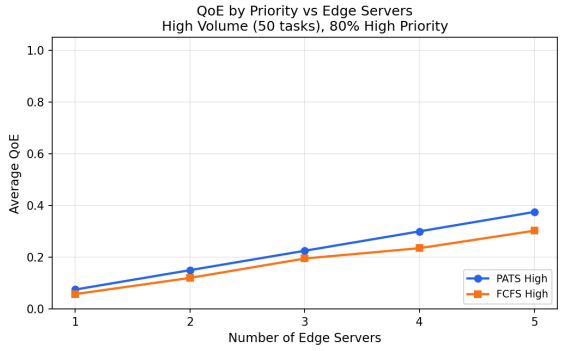
(d) Weighted Avg QoE, 40% High-Priority



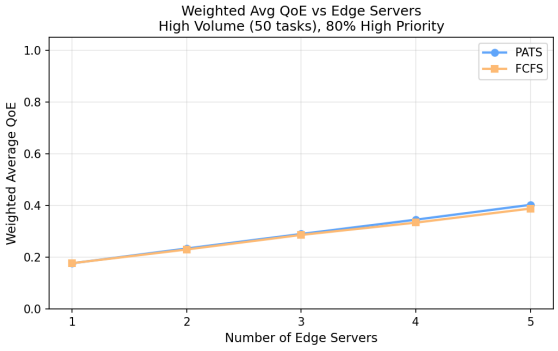
(e) QoE by Priority, 60% High-Priority



(f) Weighted Avg QoE, 60% High-Priority



(g) QoE by Priority, 80% High-Priority



(h) Weighted Avg QoE, 80% High-Priority

Figure 5.16: High Volume (50 Tasks) Results Across Priority Splits

Discussion of Results

While the high volume case displays some characteristics found in the previous two volumes, it is clearly showing that under the high task count pressure, even the optimised scheduler begins to suffer. When considering the results by high-priority percentage, what is clear that only for the low value of 20% of high-priority tasks do we see significant improvement over FCFS. Results for this volume of traffic clearly show that the improvements from PATS swiftly diminish as the high-priority percentage increases at a far more drastic rate compared to both low and medium task counts. While results are still superior as they display that even under extreme levels of pressure, PATS is still outperforming FCFS – they are far less convincing than the previous volumes. This is to be expected at high-volume task counts. What is observed is that the infrastructure is the limiting factor for PATS in these scenarios, not the algorithm itself. With 50 tasks, all submitted at one burst, the algorithm is simply restricted to the few resources it has to its disposal. When considering the scenario of 80% of tasks are labelled high-priority, the scheduler must deal with 40 high-priority tasks, all submitted at t=0. It is simply unrealistic to expect the scheduler to be able to cope as it is restricted to five edge servers which is the best case, and only a single edge server in the worst case. By observing Figure 5.16g, we can see that PATS is just about offering any sort of improvement over FCFS, however, this is an absolute worst-case for the scheduler to have to deal with. These results still speak in favour of the PATS approach as although it is under heavy stress situation and may not produce exceptional results, it is still performing better than the FCFS approach. Under more realistic scenarios of high-priority task percentages and even at high volumes, we see substantial improvement over FCFS, as the algorithm is able to execute its core ability; being able to offer the best levels of service to high-priority work among a large volume of concurrent tasks.

In terms of improvements observed with increasing energy usage, we see that predominantly, it is only with high-priority task percentages of 40% and less that we see a superior improvement compared to FCFS. This is still a notable finding as it shows how PATS is still of greater energy efficiency compared to FCFS, even at the high stress situation of 50 tasks submitted at t=0. Despite real improvements only being observed for the case of 20% and 40% of tasks being labelled high-priority, it can still be observed that rate at which PATS improves task QoE levels is far quicker than the rate of FCFS. This again is consistent with results obtained in the Section 5.2.1 and 5.2.2. The improvement of PATS is still highlighted in terms of energy consumption through its offering any available service to latency critical, high-priority tasks - even at extreme volumes of tasks.

Finally, by observing the weighted average graphs on the right of Figure 5.16, the trend found in the previous two task volumes continues. It shows that overall, although less pronounced, PATS is producing better results compared to FCFS. A finding discovered is that for the worst-case scenario, both schedulers essentially converge to produce similar results in terms of overall system performance, as seen in Figure 5.16h. While this has been remarked previously that increasing task load and increasing the percentage of work marked as high-priority, the scheduler cannot make distinctions. Though as stated previously, the purpose of the proposed algorithm was never to be able to somehow handle the worst-case scenario, under heavily constrained circumstances. Rather it was to be able to provide a select group of work which is of greater importance and tighter requirements, higher levels of service – which while not observed in the worst case, is certainly occurring in the more modest scenarios.

Overall QoE Discussion

Overall, for all three task volumes, we can see that PATS consistently achieved higher QoE scores than FCFS. A trend which stood out in all three sets of tests was the fact that PATS seemed to converge closer and closer to FCFS when increasing the percentage proportion of high-priority tasks. This as mentioned is to be expected due to the fact that PATS cannot make intelligent decisions

when the majority of tasks are of similar priority labels. Another observation when considering the results in terms of increasing task volumes is that the limiting factor at high-volume task counts is not the scheduling policy, but rather the limitation of available infrastructure. However, in scenarios where task volumes are not on the extreme scale and percentage of tasks labelled high-priority tasks only form a minority of the workload, the benefits of the PATS algorithm are most effective.

Overall Energy Discussion

The results across all three volumes reveal clearly that PATS is inherently more energy efficient when attempting to maximise high-priority task QoE. From the results, we can see that increasing the number of edge servers, for each of the different task volumes, quickly result in significantly improved QoE scores. This is in stark contrast to FCFS, which requires significantly more edge infrastructure to achieve comparable high-priority QoE levels - for example, at 20% high-priority with 25 tasks, PATS reaches maximum QoE with 2 edge servers while for FCFS, even with 5 edge servers, it only achieves a QoE of around 0.5 for high-priority tasks. This shows that PATS can substantially reduce the number of edge servers needed to maintain acceptable high-priority performance, directly lowering energy consumption and infrastructure costs. For a network operator, this is a significant finding, as it suggests that deploying fewer edge servers paired with an intelligent scheduling policy can achieve better high-priority task QoE instead of scaling up infrastructure with a naïve approach alone. This therefore allowing operators to make more informed decisions about when additional edge capacity is truly necessary versus when smarter scheduling alone can suffice. However, at higher percentages of high-priority tasks where the majority of tasks share the same priority level, the energy-QoE trade-off shifts, and the problem becomes that of edge server capacity rather than intelligent scheduling policies.

5.2.4 Varying Cloud Latency

After considering varying load volumes and different priority splits, another dimension to be varied is the cloud's latency. To conduct a set of experiments in which only the cloud latency changes, some base configuration of MEC setup must be chosen such that task volume, high-priority task percentage and number of edge servers is non-trivial and will allow for insight into how the PATS algorithm really considers the cloud as a viable backup option. For this reason, both low and medium volume of 10 and 25 tasks, respectively with 40% high-priority tasks, have been selected as the base configurations, to attempt to model a more realistic scenario. We will also consider one, two and three edge servers to monitor how increasing energy consumption helps in improving task QoE. This setup provides enough contention for edge resources that the cloud becomes a realistic scheduling option, while still maintaining a meaningful split between high and low-priority tasks. Cloud latency will vary from 25ms to 200ms, in increments of 25ms, with all other parameters held constant. The aim of these tests is to observe how increasing cloud latency impacts the scheduling decisions made by PATS compared to FCFS, and at what point the cloud latency becomes too high to be a useful offloading destination for maintaining acceptable QoE levels.

Low Volume, Varying Cloud latency

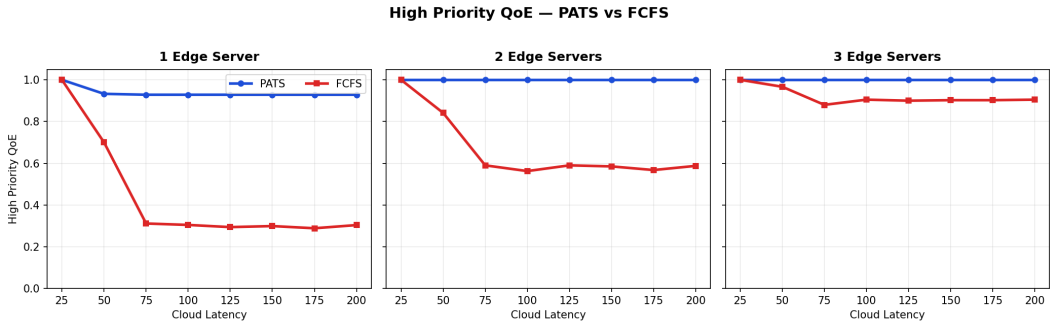


Figure 5.17: high-priority Task QoE for Low Task Volume

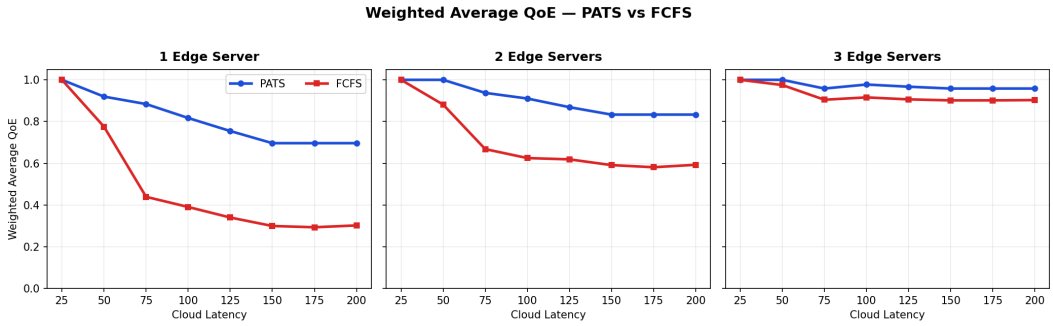


Figure 5.18: Weighted Average Task QoE for Low Task Volume

Medium Volume, Varying Cloud latency

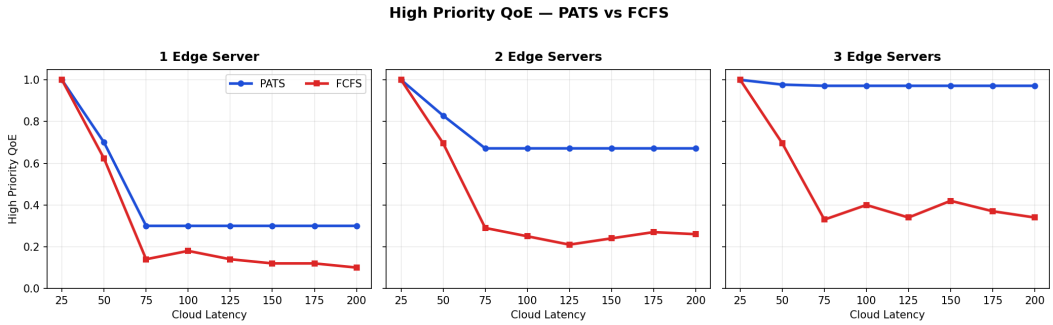


Figure 5.19: high-priority Task QoE for Medium Task Volume

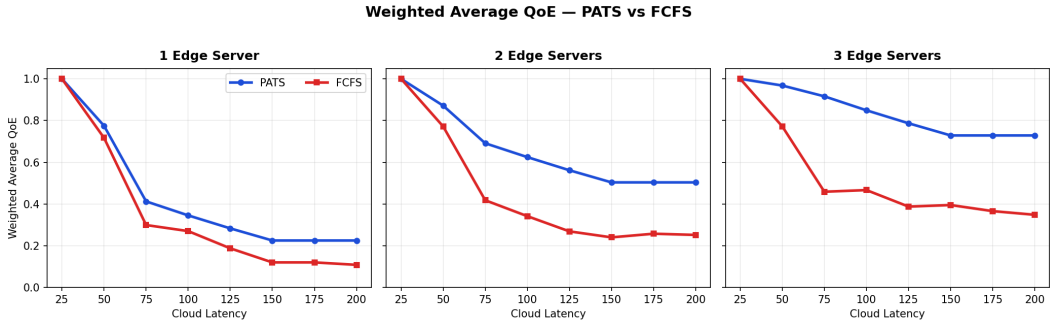


Figure 5.20: Weighted Average Task QoE for Medium Task Volume

Low Volume Discussion

Looking at high-priority QoE under low task volume in Figure 5.17, both PATS and FCFS perform similarly at low cloud latencies of 25ms, with both achieving close to 1.0 across all edge server counts. This is because at low cloud latency, the cloud effectively acts as an additional high-powered edge server with minimal latency costs, meaning even FCFS can achieve strong results. As cloud latency increases beyond 50-75ms, a sharp drop in QoE is observed for FCFS, while PATS remains significantly more resilient. For a single edge server, PATS stabilises at around 0.95 from 100ms onwards, while FCFS drops starkly to 0.3. What is being exhibited by the PATS scheduler is intelligence in recognising that it can no longer avail of the cloud for backup high-priority task execution, therefore keeps them on the edge to achieve the maximum possible QoE. With 2 edge servers, PATS can maximise high-priority task QoE, as with just 4 high-priority tasks, and 2 edge servers, all tasks are able to comfortably be executed on the edge, meeting latency requirements. By contrast, FCFS holds steady at around 0.6, while PATS achieves superior results due to its ability to allocate the best resources to the highest priority work. This also displays PATS' efficiency regarding energy, because with this configuration, the highest levels of service for the most important tasks can be provided with just 2 edge servers. At 3 edge servers, PATS is able to maximise high-priority QoE, while FCFS also performs notably well. However, this is expected due to increasing numbers of edge resources, it no longer must offload tasks to the cloud due to overly saturated edge servers. While FCFS sees significant improvement with 3 edge servers, it is still unable to maximise high-priority task QoE. Though being very close, it would suggest that FCFS would need to increase edge server count once more to meet results produced by PATS, effectively doubling the energy usage required to provide similar levels of service.

By looking at the weighted average QoE under low task volume in Figure 5.18, we can see how overall system performance, quantified using the weighted average, is affected by increasing cloud latency. The results show that PATS is in fact performing notably better than FCFS. Although there is a drop in performance compared to the results of solely high-priority task QoE, this is expected and is a continuation of the trend observed in previous tests. PATS does not see as significant of an improvement when increasing edge server count as FCFS does regarding the weighted average QoE but is due to the fact that even with one edge server, the policies integrated into PATS already protect the most important work. There is a small improvement when increasing edge server counts, however the added infrastructure is primarily being used to improve low-priority task QoE, which through the weighted average score, has a far smaller influence than improving high-priority task QoE. Therefore, as PATS improves the less important work, it sees less improvement than FCFS, which in contrast is likely using the added infrastructure to improve tasks across all priorities, which as a result, drastically improving its overall performance.

Medium Volume Discussion

Looking at high-priority QoE for the medium task volume in Figure 5.19, it's obvious that the increased task loads coupled with an increasing cloud latency have a major effect on high-priority QoE. Like the low task volume case, at just 25ms, the cloud is still acting as a high computational power backup edge resource. However, as the latency penalty increases past 50ms, results drop starkly. For one and two edge servers, there is a very pronounced drop off for high-priority QoE, due to the fact that with 10 tasks labelled high-priority and all submitted at t=0, the scheduler becomes overloaded. Beyond 75ms cloud latency, high-priority tasks offloaded to the cloud miss their deadlines entirely and contributing QoE scores of 0, causing the plateau. The QoE scores for high-priority tasks at one and two edge servers remains at around 0.3 and 0.7 respectively, as a small portion of the tasks can remain on the edge to be processed. However, the remaining tasks must be offloaded to the cloud as waiting on the edge, competing for just one or two edge servers, would take longer than just being offloaded to the cloud. However, we see massive improvements when increasing edge servers count to three. This would show that in terms of energy consumption, for medium task volumes and unacceptable cloud latency times, increasing

consumption has significant improvements on task QoE. It would be expected that the trend would follow and increasing to four edge servers under this configuration would maximise high-priority task QoE, as local edge resources have enough capacity to deal with the load. Comparing the PATS results to FCFS, we see that FCFS only shows minimal improvements in increasing edge server counts, showing that PATS protection policies of high-priority tasks are in fact benefitting performance. To see similar results for FCFS, it would seem that there would need to be enough edge resources provisioned to handle all tasks locally, which would lead to huge amounts of energy consumed. This arises simply due to its inability to distinguish tasks of different priorities, and instead blindly offloads important tasks to the cloud when edge resources are constrained.

Looking at Figure 5.20, the weighted average QoE when put under medium traffic volumes, we almost see the converse of what was seen in Figure 5.18. As mentioned, PATS saw less drastic improvements compared to FCFS for low volumes of traffic. However, for the medium traffic volumes, PATS sees greater improvements compared to FCFS. With the increasing amount of infrastructure, PATS by design will protect the important work before dealing with lesser important tasks. As stated previously, improving QoE for high-priority tasks has a much stronger influence on the weighted average due to their greater overall importance in terms of the system. FCFS by contrast, only sees marginal improvements when increasing edge server counts and overall energy consumption. This is because it distributes improved latency times across all task priorities, rather than the most important tasks. For cloud latency times of 150ms and over, PATS sees weighted QoE scores of just over 0.2 for one edge server. This more than doubles to around 0.5 when edge server counts are increased to two and jumps significantly again to just under 0.8 when increased to three. FCFS however, sees poor weighted QoE of 0.15 for one edge server, which only increases to around 0.25 when increased to two and when considering three edge servers, improves to around 0.4, which overall are much more marginal improvements compared to PATS. This illustrates more clearly than the low traffic volumes, that PATS itself sees much greater improvements when increasing energy consumption when resources are limited. What can be observed is that PATS displays a logarithmic improvement in terms of overall system performance when increasing energy use. This is due to it intelligently allocates compute to the most important work first, which sees the greatest increase in weighted average QoE. This then plateaus as edge counts continue to increase, since the algorithm shifts from protecting important work to improving QoE for low-priority work, contributing less improvement to weighted average QoE. It also highlights that to see similar results for FCFS, edge servers counts would need to be of much greater numbers than for the PATS approach, due to the way FCFS allocates resources.

5.2.5 Varying High-Priority Latency Requirements

All testing concluded thus far only utilised one value for latency requirement for high-priority work, however, this section will investigates altering this requirement. These tests are of interest because the PATS algorithm attempts to safeguard high-priority tasks by restricting the placement of low-priority tasks on edge servers, when doing so would degrade high-priority QoE. However, when latency requirements are more lenient, it would be expected that a higher number of low-priority tasks to share edge servers with high-priority ones. This set of tests also provides further insight into how the cloud can be leveraged when latency constraints allow for it, another integration of the PATS algorithm's design. These tests follow a similar configuration to that used in 5.2.4. Edge server counts of 1, 2, and 3 are again used to explore how energy resourcing affects the trade-off between task QoE and energy efficiency. The proportion of high-priority tasks is set at 40%, considering both low and medium volumes of tasks, consistent with 5.2.4, providing a meaningful volume of high-priority work without overwhelming the scheduler.

Low Volume

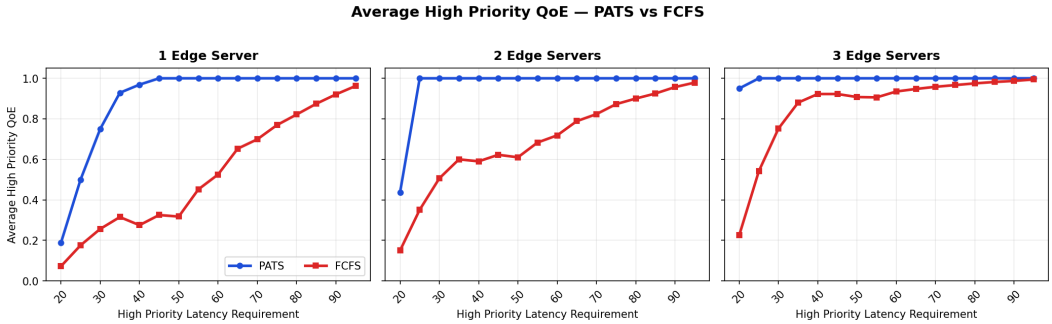


Figure 5.21: high-priority Task QoE for Low Task Volume

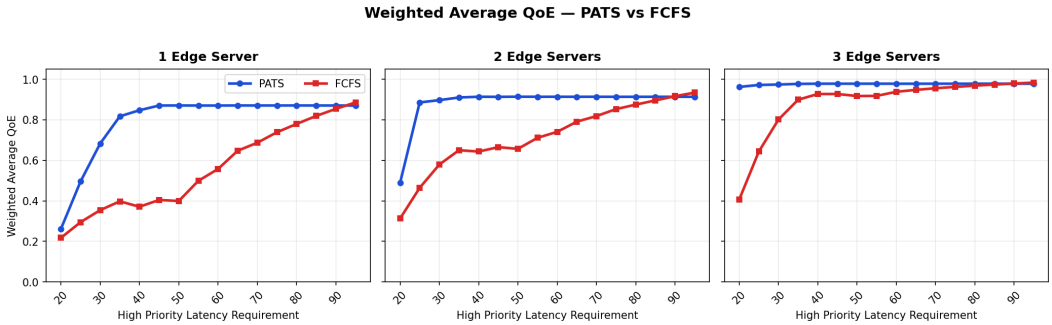


Figure 5.22: Weighted Average Task QoE for Low Task Volume

Medium Volume

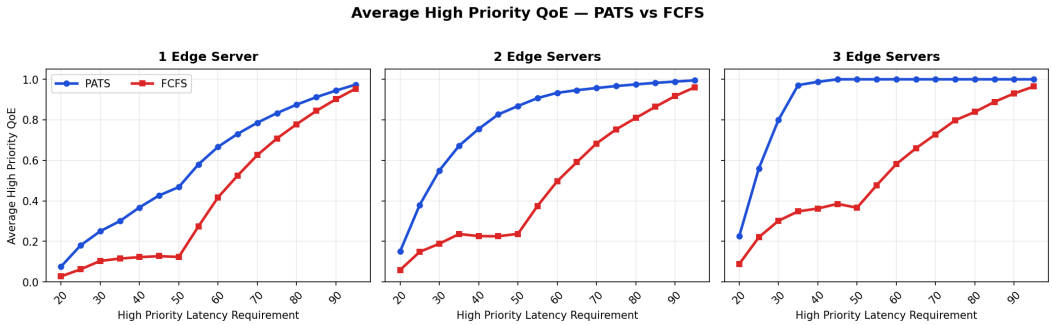


Figure 5.23: high-priority Task QoE for Medium Task Volume

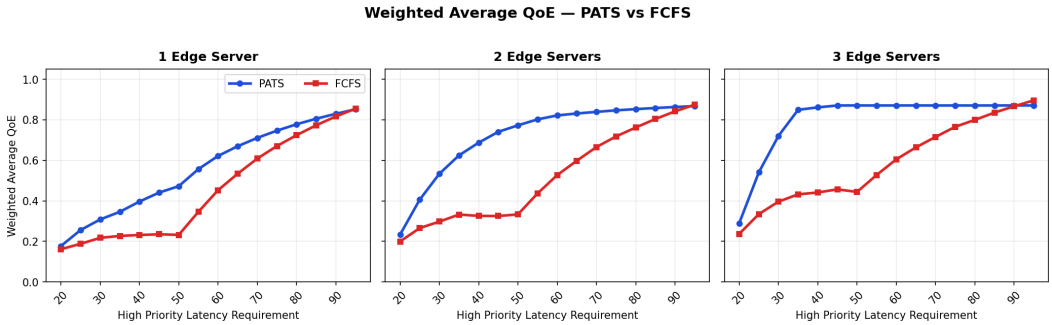


Figure 5.24: Weighted Average Task QoE for Medium Task Volume

Low Volume Discussion

Looking at Figure 5.21, we can see that PATS quickly maximises the QoE of high-priority tasks. Even for relatively short latency requirements of 35ms plus, for just one edge server, PATS can offer high levels of service, which is a notable result given the fact that tasks require at least 10ms just for execution time alone. With just two edge servers and a 25ms latency requirement, PATS can fully maximise the QoE of high-priority tasks. For 20ms of required latency, one and two edge servers produce, noticeably weak QoE scores due the incredibly tight latency requirement. However, when the edge server count is increased to three, PATS displays almost perfect QoE scores for high-priority tasks. In contrast, FCFS shows a flatter, linear improvement in QoE for high-priority tasks. Only when latency requirements are extremely lenient does FCFS show results comparable to PATS. With low task volumes and just 2 edge servers, PATS can guarantee maximum QoE for all high-priority tasks, whereas FCFS would seem to require twice as many edge servers to achieve a similar guarantee. This trade-off between energy consumption and QoE is invaluable for deployment decisions, and the results here further suggest that, with lower energy consumption, much higher service levels are achieved when utilising the PATS scheduling approach.

Figure 5.22 shows that PATS again displays superior overall task QoE compared to FCFS. An interesting point, however, is that across all three edge server counts, when high-priority latency requirements exceed 100ms, we see that FCFS actually produces better overall QoE than PATS. The reason for this is that FCFS blindly sends tasks to the cloud when edge resources are fully saturated, and once latency requirements approach 100ms, the cloud's 100ms of latency is no longer a concern. However, PATS itself will always process tasks in order of importance, which in this study is predominantly focused on priority labels, so the priority scores generated largely depend on a task's priority label. Therefore, high-priority tasks will occupy edge resources before any low-priority tasks, given their labels and the scoring function. However, low-priority tasks have a latency requirement of 50ms, which was held constant across all tests, thereby imposing tighter requirements than those of the high-priority tasks. But as the main goal of PATS under this specific scoring-function configuration is to prioritise labels, the latency requirement is superseded, leading PATS to prefer placing high-priority tasks on the edge and offloading low-priority tasks to the cloud. However, FCFS's blind cloud offloading actually helps its case in this scenario. Once high-priority latency requirements exceed the cloud round-trip time, sending high-priority tasks to the cloud no longer affects their QoE. This allows FCFS to offload some high-priority tasks while keeping low-priority tasks (with their tighter latency requirements) on the edge, which improves overall system performance. This inadvertently demonstrates that the priority scoring function of PATS is effective in protecting high-priority work. However, it also motivates future study into alternative values for the coefficients A, B, and C, which could allow low-priority tasks with tighter latency requirements to receive higher priority scores than high-priority tasks with more lenient ones.

Medium Volume Discussion

Figure 5.23 displays that for a medium volume of tasks and a single edge server, high-priority QoE rises somewhat linearly with increasing latency requirements. However, considering two and three edge servers, QoE rises much more steeply. This is an interesting find, as it reinforces the manner in which PATS improves the QoE of high-priority tasks sharply as the number of edge servers increases, similarly to as discussed in 5.2.4. Although consuming more energy, the point at which PATS maximises task QoE requires significantly less energy than FCFS. Looking at the results of the same simulations for FCFS, we also see that for high-priority latency requirements of 20-50ms, improvements are very small; however, once exceeding 50ms, improvements are drastic. This is because once the latency requirement exceeds 50ms, high-priority tasks which have been blindly offloaded to the cloud begin to contribute non-zero QoE scores. Eventually, we see it converges to almost maximal QoE, as cloud latency no longer hinders QoE scores for any task type.

Finally, in Figure 5.24 we again see a drop in QoE compared to the high-priority QoE for PATS,

and a slight increase for FCFS compared to its high-priority QoE results in Figure 5.23. Again, as mentioned, this situation arises because PATS protects important work and FCFS distributes its resources among all task types, regardless of importance. Similarly, to observed in Figure 5.22, we see that FCFS again performs better than PATS overall when considering the weighted average for high-priority latency requirements approaching 100ms. This again is for the same reason as discussed in the previous section. PATS awards its best resources to the work labelled high-priority, while FCFS sends any priority of task to the cloud when there are no locally available resources. When high-priority tasks have lenient latency requirements, greater than task round-trip time, it actually benefits the system performance as it allows for the tighter latency requirement, low-priority tasks to stay on the network edge.

5.3 Temporal Arrival Patterns

While testing concluded thus far is mainly concerned with submitting tasks at t=0 to put the system under stress to study behaviour, it does not reflect a typical MEC environment. This section focuses on looking at behaviour and performance over a longer time frame. The new abstract scheduling layer integrated into iFogSim2 makes use of the CloudSim event simulator and therefore makes it possible to simulate scenarios over various time periods, rather than simply at t=0. Three distinct arrival patterns will be studied in this section, namely, Period, Bursty, and Ramp-Up. Each pattern distributes the same workload of 100 high-priority tasks over a 1000ms window but with different arrival timings. Low-priority tasks will be submitted every 3ms to model the constant background workload typical of real-world MEC environments. The MEC environment for these tests consists of 3 edge servers and a single cloud server, using the same baseline configuration outlined in Table 5.2. By holding the total task count, priority split, and infrastructure constant, these tests isolate the effect of arrival timing on scheduling performance. Results from both PATS and FCFS will be compared to study if key design considerations truly make a difference in overall system performance.

5.3.1 Periodic-Uniform

The Periodic-Uniform pattern distributes tasks evenly across the simulation window by submitting 20 batches of 5 tasks every 50ms. Each batch contains 5 high-priority tasks, while the low-priority, background traffic are being submitted consistently every 3ms. This pattern models a steady workload, which could be found in a network of IoT sensors reporting data at fixed intervals or a set of users generating requests at a constant rate. The system never experiences high pressure and with small discrete workloads arriving, it gives the scheduler ample time to deal with tasks batch by batch. This arrival pattern is the most favourable of the three and should keep queue pressure to a minimum.

5.3.2 Bursty

The Bursty arrival pattern delivers the same 100 high-priority tasks in 8 concentrated bursts of 12 to 13 tasks each, spaced 125ms apart. Unlike the Periodic-Uniform pattern, this creates periods of high demand followed by periods of low demand. This pattern is representative of real-world scenarios such as event-driven IoT networks, where a triggering event causes a sudden spike in task submissions. This pattern will assess the schedulers' performance in dealing with high bursts of traffic under resource constrained environments and being able to recover quickly by making

informed placement and offloading decisions to ensure of maximum task QoE.

5.3.3 Ramp-up

The Ramp-Up pattern models a gradually increasing workload, starting with small batches of approximately 2 high-priority tasks and growing to batches of around 15 tasks, submitted at 100ms intervals across the simulation window. This models scenarios in which MEC environments see gradual deployment of IoT sensors for example. At the beginning of the simulation, edge servers are primarily dealing with the background tasks being constantly generated, and the scheduler can comfortably execute all batches of high-priority tasks locally. As batch sizes grow, the scheduler must increasingly rely on intelligent placement, cloud offloading and migration to maintain QoE for high-priority tasks. This pattern essentially allows for evaluation on how the PATS algorithm handles progressively worsening conditions.

Arrival Patterns

Below is a graphical summary of the three arrival patterns over the 1000ms time window:

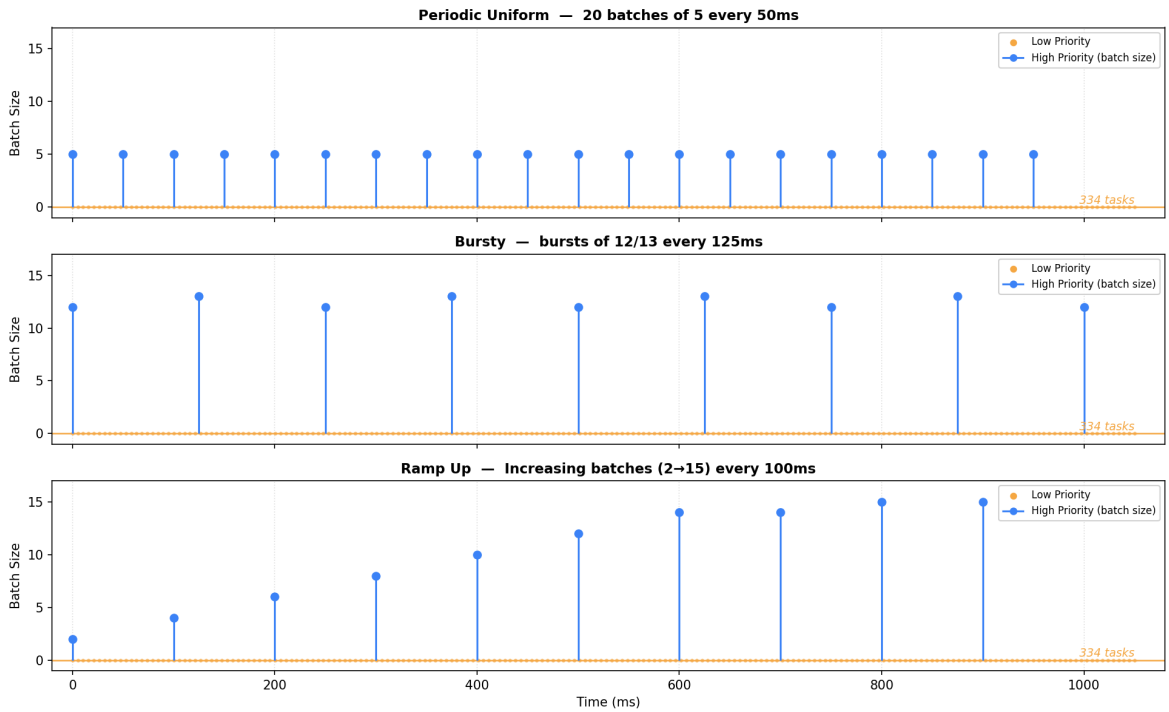


Figure 5.25: Periodic Uniform, Bursty and Ramp Up Arrival Patterns

5.3.4 Results

Figure 5.26 presents the QoE results across all three arrival patterns for both the PATS and FCFS schedulers. The results demonstrate that PATS consistently outperforms FCFS across all arrival patterns, though the magnitude of the improvement varies with the pattern.

QoE Metrics

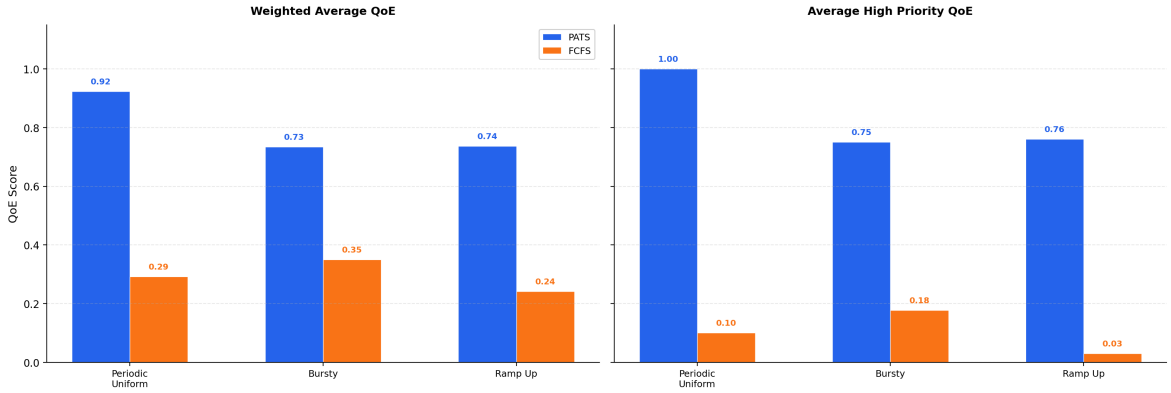


Figure 5.26: QoE Metrics for Different Arrival Patterns

With the Periodic-Uniform pattern, PATS achieves its strongest results, with a weighted average QoE of 0.92 compared to just 0.29 for FCFS. Average high-priority QoE reaches the maximum score of 1.0 for PATS, meaning that every high-priority task met its latency requirement, while FCFS manages only 0.1. This result is expected given the favourable conditions of the Periodic-Uniform pattern: the steady, predictable flow of tasks allows the PATS scheduler to consistently place high-priority tasks on the least loaded edge servers and offload low-priority tasks to the cloud when necessary, without ever being overwhelmed by sudden demand. The FCFS scheduler lacks priority awareness and due to the buildup of the background traffic, it cannot differentiate between low and high-priority, thus leaving high-priority tasks to wait in the queue while it deals with existing low-priority tasks which arrived earlier. This leads to an average of 0.1 for high-priority task QoE, a far more inferior result compared to PATS. The integration of priority-based preemption/migration into PATS shows its greatest advantage here, as it is able to free up enough edge servers to deal with the small batches of high-priority tasks being submitted periodically, yielding far superior QoE results. Looking at the weighted average, we can see that the periodic, uniform nature of the workload means PATS does not have to make significant sacrifices or resort to aggressive cloud offloading to handle low-priority tasks.

The Bursty pattern presents a more challenging scenario compared to the periodic-uniform scenario. PATS achieves a weighted average QoE of 0.73, with an average high-priority QoE of 0.75. While results are lowered compared to the Periodic-Uniform, they still display a substantial improvement over FCFS which achieves a weighted average QoE of only 0.35 and an average high-priority QoE of 0.18. During each burst, 12 to 13 high-priority tasks enter the queue, and with only 3 edge servers, the scheduler cannot immediately place all high-priority tasks on local edge servers. This forces the PATS algorithm to more aggressively utilise its migration and cloud offloading mechanisms. Despite this, the priority scoring function ensures that high-priority tasks are always considered first within each scheduling cycle. Interestingly, FCFS performance actually improves under this arrival pattern compared to the Periodic-Uniform case. This can be attributed to the concentrated nature of the bursts. Under the Periodic-Uniform pattern, high-priority tasks arrive in small batches of 5, distributed with the constant stream of low-priority background tasks arriving every 3ms. As FCFS processes tasks strictly in arrival order, low-priority tasks that arrive between batches are scheduled ahead of the next batch of high-priority tasks, repeatedly blocking them from timely execution. Under the Bursty pattern, however, 12 high-priority tasks arrive together in a single burst. This results in the scheduler encountering them consecutively in the queue before low-priority background tasks have had time to accumulate between them. In effect, the concentrated arrivals inadvertently group high-priority tasks together in the FCFS queue, allowing more of them to be processed in sequence without low-priority tasks interleaving and causing delays.

Considering the Ramp-Up pattern, PATS achieves a weighted average QoE of 0.74 and an average

high-priority QoE of 0.78. These results are more comparable to the Bursty pattern, though with a slightly higher high-priority QoE. This is an interesting finding, as although the Ramp-Up pattern ultimately generates larger individual batches than the Bursty pattern, the gradual increase in load gives the PATS scheduler time to establish efficient placement decisions early in the simulation. High-priority tasks arriving in the initial small batches are processed quickly, freeing edge resources before the larger batches arrive. This temporal advantage means that by the time the system faces its heaviest load, the scheduler has already cleared earlier work and can dedicate more resources to the incoming high-priority tasks. FCFS, by contrast, achieves an average high-priority QoE of only 0.03 under this pattern, which was its worst result across all three patterns. This poor performance occurs because as batch sizes grow, the FCFS scheduler becomes overwhelmed. The increasing volume of high-priority tasks progressively increases the number of tasks waiting in queues and with the quicker rate of batch submissions, leads to high-priority tasks being sent to the cloud as the scheduler cannot deal with the backlog intelligently.

Operational Metrics

Figure 5.27 presents some key operational metrics collected for both schedulers under the various arrival patterns. These results offer a deeper understanding of the scheduling decisions made by both PATS and FCFS

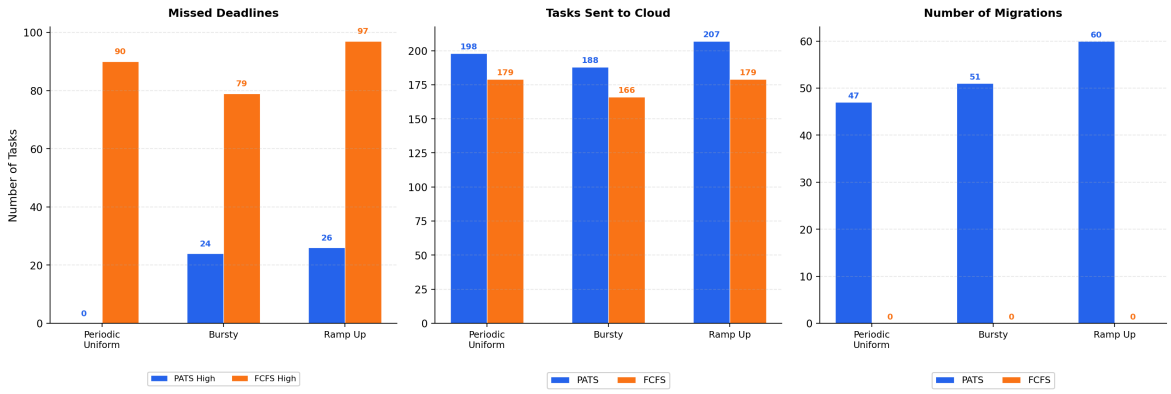


Figure 5.27: Operational Metrics

The missed deadline metric counts the number of tasks that failed to complete within their required latency, resulting in tasks not achieving the maximum QoE score of 1.0. This metric reveals the most significant difference between the two schedulers. Under the PATS algorithm, no high-priority tasks miss their deadlines in the Periodic-Uniform pattern. For the Bursty and Ramp-Up patterns, approximately only a quarter fall short of a QoE score of 1.0, which is expected given the increased system load under these conditions. Importantly, as discussed in Section 4.6 regarding QoE value definitions, the 24 and 26 high-priority tasks affected in the Bursty and Ramp-Up patterns respectively, did not necessarily receive a poor level of service rather; they did not achieve the maximum satisfaction score. For the majority of applications, the practical difference between a QoE of 1.0 and 0.9 may have no adverse effect on the end user. Nevertheless, this metric is useful for quantifying how many tasks contribute to a drop in overall high-priority QoE. These results are a direct consequence of the priority scoring function and the tiered placement strategy, which ensure that high-priority tasks are always considered first and placed on servers capable of meeting their latency requirements. Low-priority tasks under PATS do however experience missed deadlines across all patterns, reflecting the algorithm's decision to delay or offload these tasks in favour of higher-priority work. In contrast, FCFS produces significantly more missed deadlines for high-priority tasks: approximately 90 under Periodic-Uniform, 79 under Bursty, and 97 under Ramp-Up. These figures indicate that under FCFS, the majority of high-priority tasks fail to meet their latency requirements, which is a result that would be unacceptable in the context of

time-critical applications such as health monitoring.

The number of tasks offloaded to the cloud is broadly similar between the two schedulers, with PATS sending approximately 198 tasks under Periodic-Uniform, 188 under Bursty, and 207 with Ramp-up while FCFS sends approximately 179, 166 and 179, respectively. However, the critical difference lies in which tasks are sent to the cloud. The PATS algorithm selectively offloads low-priority tasks to the cloud when doing so frees edge capacity for high-priority work, or when the cloud can deliver acceptable QoE for the offloaded task. There is a direct correlation between the number of tasks sent to the cloud for PATS and the number of tasks which miss their deadline. We can see that by sending tasks to the cloud, it will result in missing latency requirements due to the cloud round-trip cost. However, this is a necessary action to ensure that higher-priority tasks are meet their requirements, which means they must stay on the network edge rather than being sent to the cloud. The PATS scheduler correctly identifies that offloading low-priority tasks with more lenient QoE drop-off to the cloud instead of high-priority tasks, will result in a better overall system performance. As mentioned before, the core objective of PATS is to maintain high levels of service for high-priority tasks, which are more latency-sensitive, and this is exactly what it is doing, by being more selective in its decisions and weighing out what course of actions would bring about the best results. FCFS however, lacks this selectivity, and offloads tasks based purely on edge capacity constraints without considering priority. This means that with FCFS, high-priority tasks may be sent to the cloud simply because they arrived after a sequence of low-priority tasks that filled the edge servers, resulting in the high-priority tasks incurring the full cloud round-trip latency penalty.

The migration metric highlights a capability unique to the PATS algorithm. PATS triggers 47 migrations under Periodic-Uniform, 51 under Bursty, and 60 under Ramp-Up. FCFS, by design, performs zero migrations across all patterns. The increasing number of migrations from Periodic-Uniform to Ramp-Up is consistent with the increasing resource pressure of each pattern. Under the Ramp-Up pattern, where batch sizes grow progressively, the scheduler encounters more situations in which a high-priority task arrives, and all edge servers are occupied, requiring the preemption and reassignment of a low-priority task. The migration numbers align with expectations and somewhat mirror the arrival patterns. Periodic uniform requires the least amount as when the system is dealing with steady, predictable arrivals, it rarely creates the kind of contention that necessitates migration. Looking at the Ramp-Up, with the increasing sizes of batches, it results in the scheduler having to reallocate resources dynamically, as the system comes under more stress. This metric shows that priority-based preemption/migration is a great aid in ensuring that latency-sensitive workloads, which are of higher priority, achieve optimal quality of service.

Chapter 6: Conclusions and Future Work

6.1 Conclusion

This project implemented a new purpose-built scheduling layer within the iFogSim2 simulator, allowing future researchers to define their own scheduling policies and study performance within a controlled environment. The motivation for the new scheduling layer stems from the fact that iFogSim2, while modelling many advanced concepts, predefined entities make it difficult to alter. The new integrations into the simulator allow users to study the core issue of task scheduling within MEC environments.

The project looked into defining a new priority-aware task scheduling algorithm for MEC environments, which aimed to maintain latency-sensitive, high-priority task QoE high while also attempting to make intelligent scheduling decisions to benefit overall system performance. Using the new scheduling framework, two algorithms were implemented, FCFS for baseline comparison purposes, and the newly designed PATS algorithm. Both algorithms were put under extensive testing, including verification, performance and temporal arrival pattern testing.

The results of this project showed that across a wide range of performance tests which considered various dimensions, and across three distinct arrival patterns, PATS consistently outperformed the FCFS baseline, protecting high-priority task QoE and doing so with fewer edge resources. The project has therefore successfully achieved its core aim of demonstrating that priority-aware, heuristic scheduling can meaningfully improve service quality and energy efficiency in MEC environments, while laying the groundwork for further research through the new scheduling framework.

6.2 Future Work

6.2.1 Task Heterogeneity

While this study assessed both schedulers under a multi-dimensional testing regime, heterogeneity within specific task classes remains unexplored. Task requirements stayed constant for all tasks of a given priority label throughout the simulations, leaving room for future work in scenarios where multiple high-priority tasks each have different latency requirements. This would likely require altering the policies defined in PATS - for example, the `resolveConflict` method could be extended to preempt high-priority tasks with lenient latency requirements in favour of high-priority tasks with tighter latency requirements. Future studies could also compare different values for the scoring function coefficients A, B, and C, shifting the scheduler from being purely priority-focused to a blend of priority and latency awareness. A potential side effect is that lower priority tasks may actually be considered ahead of high-priority tasks due to tighter latency budgets.

Another direction is the addition of further task labels. This study focused on two priority classes, but intermediate labels, or even application-based labels rather than a low/high hierarchy, could be explored as a more expressive way to model task importance.

6.2.2 Application Modelling

Building on the idea of application-based labels, generalising the small discrete tasks used in this study into full applications is another possible direction. Modelling applications which generate a stream of tasks would allow for per-application QoE monitoring, a higher level of abstraction than what is considered in this project. This would also open up the possibility of studying dependencies between tasks within a single application, where the completion of one task may influence the scheduling priority of subsequent tasks in the pipeline.

6.2.3 Priority Label Trust and Admission Control

As noted previously, this study assumes that the priority label attached to each incoming task is trusted by the scheduler. In a real deployment this assumption would not hold, as clients have a clear incentive to label all of their tasks as high-priority to receive the best levels of service, which would erode the benefits of priority-aware scheduling entirely. A natural extension would be to integrate an admission control layer that governs which clients may submit high-priority tasks, along with per-client rate limits or trust-based mechanisms such as attestation from certified device classes. Tasks from trusted sources such as medical monitors or automotive safety systems could then be treated with greater confidence than those from arbitrary applications.

6.2.4 Energy Efficiency

As discussed, PATS is additionally more energy efficient than FCFS, as it can offer higher levels of service with fewer edge servers provisioned. A natural extension would be to make the scheduler explicitly energy-aware by dynamically spinning edge servers up or down based on task load. The data collected in this study already provides insight into how many servers are needed to maximise QoE at different task volumes, which could serve as a starting point for such a mechanism and would improve PATS' context-awareness over long-term simulations.

6.2.5 Other Scheduling Policies

The new scheduling framework for iFogSim2 makes it straightforward to add new algorithms and compare them against FCFS or PATS. Integrating other priority-aware scheduling algorithms would allow for a more thorough analysis of PATS and would further verify the benefits of its specific scheduling policies.

6.2.6 Other Metrics

This study makes use of high-priority QoE averages and weighted QoE averages, but reporting additional metrics such as worst-case QoE could be an interesting future approach. The integrated `SchedulerMetrics` class already tracks minimum QoE per priority label, so studying these results is a natural next step.

Acknowledgements

I would like to express my sincere thanks to my supervisor, Professor Liam Murphy for his guidance and support throughout the duration of the project. I'm extremely grateful for his expertise and insight which were instrumental in the development of this work. I would also like to thank my family and friends for their continued support and encouragement throughout the year.

Bibliography

1. IoT Analytics. *State of IoT 2025: Number of connected IoT devices growing 14% to 21.1 billion globally* 2025. https://iot-analytics.com/number-connected-iot-devices/.
2. Liu, S. *et al.* Edge Computing for Autonomous Driving: Opportunities and Challenges. *Proceedings of the IEEE* **107,** 1697–1716 (2019).
3. ETSI. *MEC in 5G Networks* tech. rep. White Paper No. 28 (European Telecommunications Standards Institute, 2018).
4. Mell, P. & Grance, T. *The NIST Cloud Computing Definition* tech. rep. (National Institute of Standards and Technology, 2011).
5. Buyya, R., Yeo, C. S., Venugopal, S., Broberg, J. & Brandic, I. Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility. *Future Generation computer systems* **25,** 599–616 (2009).
6. Armbrust, M. *et al.* A view of cloud computing. *Communications of the ACM* **53,** 50–58 (2010).
7. Satyanarayanan, M. The Emergence of Edge Computing. *Computer* **50,** 30–39 (2017).
8. Mach, P. & Becvar, Z. Mobile edge computing: A survey on architecture and computation offloading. *IEEE communications surveys & tutorials* **19,** 1628–1656 (2017).
9. ETSI, M. Multi-access edge computing (mec) framework and reference architecture. *ETSI GS MEC* **3** (2019).
10. Taleb, T. *et al.* On Multi-Access Edge Computing: A Survey of the Emerging 5G Network Edge Cloud Architecture and Orchestration. *IEEE Communications Surveys & Tutorials* **19,** 1657–1681 (2017).
11. Abbas, N., Zhang, Y., Taherkordi, A. & Skeie, T. Mobile Edge Computing: A Survey. *IEEE Internet of Things Journal* **5,** 450–465 (2018).
12. Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M. & Ayyash, M. Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications. *IEEE Communications Surveys & Tutorials* **17,** 2347–2376 (2015).
13. Luo, Q., Hu, S., Li, C., Li, G. & Shi, W. Resource scheduling in edge computing: A survey. *IEEE communications surveys & tutorials* **23,** 2131–2165 (2021).
14. Silberschatz, A., Galvin, P. B. & Gagne, G. *Operating system concepts, windows XP update* (John Wiley & Sons, 2006).
15. Tanenbaum, A. *Modern operating systems* (Pearson Education, Inc., 2009).
16. Abohamama, A. S., El-Ghamry, A. & Hamouda, E. Real-time task scheduling algorithm for IoT-based applications in the cloud–fog environment. *Journal of Network and Systems Management* **30,** 54 (2022).
17. Islam, A., Debnath, A., Ghose, M. & Chakraborty, S. A Survey on Task Offloading in Multi-access Edge Computing. *Journal of Systems Architecture* **118,** 102225 (2021).
18. Wang, S., Xu, J., Zhang, N. & Liu, Y. A survey on service migration in mobile edge computing. *IEEE Access* **6,** 23511–23528 (2018).
19. Singh, R., Sukapuram, R. & Chakraborty, S. A survey of mobility-aware multi-access edge computing: Challenges, use cases and future directions. *Ad Hoc Networks* **140,** 103044 (2023).

20. The Linux Kernel Community. *CFS Scheduler — The Linux Kernel Documentation* Linux Kernel Documentation. 2007. https://docs.kernel.org/scheduler/sched-design-CFS.html.

21. Fahad, M., Shojafar, M., Abbas, M., Ahmed, I. & Ijaz, H. A multi-queue priority-based task scheduling algorithm in fog computing environment. *Concurrency and Computation: Practice and Experience* **34,** e7376 (2022).

22. Xie, R. *et al.* Priority-aware task scheduling in computing power network-enabled edge computing systems. *IEEE Transactions on Network Science and Engineering* (2025).

23. Mahmud, R., Pallewatta, S., Goudarzi, M. & Buyya, R. iFogSim2: An extended iFogSim simulator for mobility, clustering, and microservice management in edge and fog computing environments. *Journal of Systems and Software* **190,** 111351 (2022).

24. Gupta, H., Vahid Dastjerdi, A., Ghosh, S. K. & Buyya, R. iFogSim: A Toolkit for Modeling and Simulation of Resource Management Techniques in Internet of Things, Edge and Fog Computing Environments. *Software: Practice and Experience* **47,** 1275–1296 (2017).

25. Calheiros, R. N., Ranjan, R., Beloglazov, A., De Rose, C. A. F. & Buyya, R. CloudSim: A Toolkit for Modeling and Simulation of Cloud Computing Environments and Evaluation of Resource Provisioning Algorithms. *Software: Practice and Experience* **41,** 23–50 (2011).

26. Kleinrock, L. Time-shared systems: A theoretical treatment. *Journal of the ACM (JACM)* **14,** 242–261 (1967).

27. Li, C., Ding, C. & Shen, K. *Quantifying the Cost of Context Switch* in *Proceedings of the ACM Workshop on Experimental Computer Science* (2007), 2–es.

28. Fan, X., Weber, W.-D. & Barroso, L. A. Power provisioning for a Warehouse-sized Computer. *ACM SIGARCH computer architecture news* **35,** 13–23 (2007).

29. Sheng, S., Chen, P., Chen, Z., Wu, L. & Yao, Y. Deep Reinforcement Learning-Based Task Scheduling in IoT Edge Computing. *Sensors* **21,** 1666. https://doi.org/10.3390/s21051666 (2021).

30. Mao, Y., You, C., Zhang, J., Huang, K. & Letaief, K. B. A survey on mobile edge computing: The communication perspective. *IEEE communications surveys & tutorials* **19,** 2322–2358 (2017).

31. Machen, A., Wang, S., Leung, K. K., Ko, B. J. & Salonidis, T. Live service migration in mobile edge clouds. *IEEE Wireless Communications* **25,** 140–147 (2017).

32. Hartmann, M., Hashmi, U. S. & Imran, A. Edge computing in smart health care systems: Review, challenges, and research directions. *Transactions on Emerging Telecommunications Technologies* **33,** e3710 (2022).

33. Aggarwal, S. *et al. Fully decentralized computation offloading in priority-driven edge computing systems* in *IEEE INFOCOM 2025-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)* (2025), 1–8.

Appendix A: GitLab Repository & Simulation Videos

The fork of the original iFogSim2 repository, extended as part of this study, can be found here: GitLab Repository.

Video recordings of the verification test scenarios discussed in Section 5.1 being run can also be accessed by clicking on the hyperlinks provided below.

- Test 1: Priority-Aware Task Ordering
- Test 2: Load-Aware Edge Server Selection
- Test 3: Default Isolation of High-Priority Tasks
- Test 4: Permitted Co-location When QoE is Preserved
- Test 5: Rejected Co-location Under QoE Risk
- Test 6: Preemption to Protect High-Priority QoE
- Test 7: Cloud Offload for Latency-Tolerant Low-Priority Tasks